Numeric range algorithms

Document #:	P3732R2 [Latest] [Status]
Date:	2026-03-26
Project:	Programming Language C++
Audience:	SG1,SG9

Reply-to:	Ruslan Arutyunyan <ruslan.arutyunyan@intel.com> Mark Hoemmen <mhoemmen@nvidia.com> Alexey Kukanov <alexey.kukanov@intel.com>

Contents

§ Abstract

We propose `ranges` algorithm overloads (both parallel and non-parallel) for the `<numeric>` header.

§ 1 Revision history

§ 1.1 R0

SG1 reviewed R0 during the Sofia meeting with the following feedback:

— SG1 agrees that users should have a way to specify an identity value. SG1 asks whether there is any need to specify this as a compile-time value, or whether a run-time-only interface would suffice. One concern is the potential cost of broadcasting an identity value at run time to all threads, versus initializing each thread's accumulator to a value known at compile time.

SF	F	N	A	SA
4	5	1	0	0

— SG1 has no objection to adding `transform_*` variants of algorithms.

— SG1 asks us to add `reduce_into` and `transform_reduce_into`, that is, versions of `reduce` and `transform_reduce` that write the reduction result to an output range of one element. (We asked SG1 to take this poll because LEWG rejected an analogous design for `std::linalg` reduction-like algorithms such as dot product and norms.)

SF	F	N	A	SA
4	4	0	0	0

— SG1 members would like separate proposals on fixing *`movable-box`* trivial copyability, and fixing performance issues with views in general.

§ 1.2 R1

— Add `reduce_into`, `sum_into`, and `product_into` algorithms

— Revise non-wording sections

  — Explain `reduce_into` and `transform_reduce_into`, as well as `sum_into`, `product_into`, and `dot_into` as special cases

  — Show different designs for specifying an identity value

  — Conclude that `reduce_first` is not needed

§ 1.3 R2

— Restructure a paper to focus more on what we propose and move the details below

— Add API overview

— Add refined design for operation identity

§ 1.3.1 SG1 polls for R1

Poll 1: P3732R1 “Numeric Range Algorithms” `_into` optimization is valuable.

SF	F	N	A	SA
4	5	0	0	0

Outcome: Unanimous consent.

Poll 2: P3732R1 “Numeric Range Algorithms” operator identity optimization is valuable.

SF	F	N	A	SA
3	6	0	0	0

Outcome: Unanimous consent.

§ 1.3.2 SG9 polls for R1

Poll 1: We want `reduce(in, init, [](auto lhs, auto rhs) { ... })` to work without providing an identity for the binary operation. The algorithm then doesn’t use an identity.

Outcome: No objection to unanimous consent.

Poll 2: We want `reduce(in, init, std::ranges::plus{})` (or `multiplies`, etc.) to work using the natural identity of the operation (at least for built-in types, possibly more in a TBD mechanism) (by augmenting the definition of the standard library function objects with the identity somehow).

Outcome: No objection to unanimous consent.

Poll 3: To associate an ad-hoc lambda with an identity, we prefer the option `reduce(in, init, binary_operation([](auto lhs, auto rhs) { ... }, identity))` over the options `reduce(in, init, [](auto lhs, auto rhs) { ... }, op_identity(identity))` or `reduce(in, init, [](auto lhs, auto rhs) { ... }, identity)`.

SF	F	N	A	SA
4	3	0	0	0

Outcome: Strong consensus in favor.

Poll 4: We want the serial version of `reduce_into` (and variants) for consistency now.

Outcome: No objection to unanimous dissent.

Poll 5: We want (the parallel) `reduce_into` to take a range as output instead of an output iterator (like all the other parallel range algorithms).

Outcome: No objection to unanimous consent.

Poll 6: We want projections for the algorithms proposed by P3732R1 "Numeric Range Algorithms".

SF	F	N	A	SA
0	0	2	4	2

Outcome: Strong consensus against.

§ 2 Motivation

C++20 added ranges version of algorithms. Unfortunately, only algorithms from `<algorithm>` and `<memory>` headers were added; not from `<numeric>` header. Also, all the added algorithms are non-parallel.

Later C++23 added more algorithms to `ranges` namespace including `ranges::iota` to `<numeric>` header and `ranges::fold_` algorithm family, which is in `<algorithm>` header but could arguably be in `<numeric>` header as well, since it is very close to `reduce` or `accumulate`. No parallel algorithms in namespace `ranges` for C++23 timeframe, though.

The good news is C++26 finally adds Parallel Range Algorithms (see [P3179R9]) for the existing algorithms in `ranges` namespace, where applicable. Still we are missing `<numeric>` algorithms that are extremely useful for parallelism and important for HPC use cases, like `reduce` and `scan`. Thus, this paper aims to address this gap by adding both parallel and non-parallel numeric range algorithms, which make the most sense from the authors perspective.

§ 3 Proposal summary

We propose `ranges` overloads (both parallel and non-parallel) of the following algorithms:

— `reduce`, unary `transform_reduce`, and binary `transform_reduce`;

— `inclusive_scan` and `transform_inclusive_scan`; and

— `exclusive_scan` and `transform_exclusive_scan`.

These correspond to existing algorithms with the same names in the `<numeric>` header. Therefore, we called them "numeric range(s) algorithms."

We also propose adding "`_into`" versions of `reduce` and `transform_reduce`, that write the reduction result into a sized range.

Finally, we propose parallel and non-parallel convenience wrappers:

— `ranges::sum` and `ranges::product` for special cases of `reduce` with addition and multiplication, respectively;

— `ranges::dot` for the special case of binary `transform_reduce` with transform `multiplies{}` and reduction `plus{}`; and

— `ranges::sum_into`, `ranges::product_into`, and `ranges::dot_into` (the "`_into`" versions of `sum`, `product`, and `dot`).

The following sections explain why we propose these algorithms and not others. This relates to other aspects of the design besides algorithm selection, such as whether to include optional projection parameters. For more information please look at Scope rationale and design.

§ 3.1 API overview

```
// Non-parallel overloads of reduce

template<forward_iterator I, sized_sentinel_for<I> S, class T =
        iter_value_t<I>,
        indirectly-binary-foldable<T, I> F>
constexpr auto reduce(I first, S last, T init, F binary_op);

template<sized-forward-range R, class T = range_value_t<R>,
         indirectly-binary-foldable<T, iterator_t<R>> F>
constexpr auto reduce(R&& r, T init, F binary_op);

template<forward_iterator I, sized_sentinel_for<I> S, indirectly-binary-
        foldable<I, I> F>
constexpr auto reduce(I first, S last, F binary_op);

template<sized-forward-range R, indirectly-binary-foldable<T, itera-
        tor_t<R>> F>
constexpr auto reduce(R&& r, F binary_op);

// Parallel overloads of reduce

template<execution-policy Ep, random_access_iterator I, sized_sentinel_-
        for<I> S,
        class T = iter_value_t<I>, indirectly-binary-foldable<T, I> F>
auto reduce(Ep&& exec, I first, S last, T init, F binary_op);

template<execution-policy Ep, sized-random-access-range R,
         class T = range_value_t<R>, indirectly-binary-foldable<T, itera-
        tor_t<R>> F>
auto reduce(Ep&& exec, R&& r, T init, F binary_op);

template<execution-policy Ep, random_access_iterator I, sized_sentinel_-
        for<I> S,
        indirectly-binary-foldable<I, I> F>
auto reduce(Ep&& exec, I first, S last, F binary_op);

template<execution-policy Ep, sized-random-access-range R,
```

```
        indirectly-binary-foldable<iterator_t<R>, iterator_t<R>> F>
auto reduce(Ep&& exec, R&& r, F binary_op);

// Parallel overloads of reduce_into

template<execution-policy Ep, random_access_iterator I, sized_sentinel_-
        for<I> S,
         forward_iterator O, sized_sentinel_for<O> OS, indirectly-binary-
        foldable<I, I> F>
auto reduce_into(Ep&& exec, I in_first, S in_last, O out_first, OS out-
        _last, F binary_op);

template<execution-policy Ep, sized-random-access-range I, sized-forward-
        range OutR,
        indirectly-binary-foldable<iterator_t<I>, iterator_t<I>> F>
auto reduce_into(Ep&& exec, I&& in_range, OutR&& out_range, F binary_op);

// Non-parallel overloads of exclusive scan

template<forward_iterator I, sentinel_for<I> S, forward_iterator O, sen-
        tinel_for<O> OutS,
         class T = iter_value_t<I>, indirectly-binary-foldable<I, T> F>
constexpr in_out_result<I, O> exclusive_scan(I first, S last, O result,
        OutS result_last,
                                             T init, F op);

template<forward_range R, forward_range OutR, class T = iter_value_t<I>,
         indirectly-binary-foldable<iterator<I>, T> F>
constexpr                    in_out_result<ranges::borrowed_iterator_t<R>,
        borrowed_iterator_t<O>>
  exclusive_scan(R&& result, OutR&& result_last,
                 T init, F op);

// Parallel overloads of exclusive scan

template<execution-policy Ep, random_access_iterator I, sized_sentinel_-
        for<I> S,
         random_access_iterator O, sized_sentinel_for<O> OutS,
         class T = iter_value_t<I>,
         indirectly-binary-foldable<I, T> F>
in_out_result<I, O> exclusive_scan(I first, S last, O result, OutS
        result_last,
                                   T init, F op);

template<execution-policy Ep, sized-random-access-range R,
         sized-random-access-range OutR, class T = iter_value_t<I>,
         indirectly-binary-foldable<iterator<I>, T> F>
constexpr                    in_out_result<ranges::borrowed_iterator_t<R>,
        borrowed_iterator_t<O>>
  exclusive_scan(Ep&& exec, R&& result, OutR&& result_last,
                 T init, F op);
```

§ 3.2 Operation identity

Based on our experience for C++17 Numeric Parallel Algorithms we allow to pass an optional operation identity (neutral to binary operation element) to Numeric Parallel Range

Algorithms: to *reduce and *scan family, in particular. We propose that users are able to attach operation identity to a binary operation. The value can be known either at compile-time or at run-time.

An algorithm knows how to check whether the identity is available. Essentially, the rules are the following:

— Read a run-time value of an operation identity if the expression is well-formed,

— Otherwise, read a compile-time value operation identity if this expression is well-formed,

— ~~Otherwise, try to default construct operation identity from input range value type if this expression is well-formed,~~

— Otherwise, an algorithm proceeds without an operation identity value.

The rules above can be applied to any binary operation, thus we propose to add compile-time known identity to binary operations in the standard, where it makes sense. For example:

```
template <typename T = void>
struct plus
{
    template <std::same_as<T> TT> // add constraints that TT is fundamen-
        tal type
    static constexpr auto operation_identity = TT(0);

    T operator()(const T& t1, const T& t2) const
    {
        return std::plus<T>{}(t1, t2);
    }
};

template <>
struct plus<void>
{
    template <typename IdentityType> // add constraints that IdentityType
        is fundamental type
    static constexpr auto operation_identity = IdentityType(0);

    template <typename T1, typename T2>
    decltype(auto) operator()(T1&& t1, T2&& t2) const
    {
        return std::plus<>{}(std::forward<T1>(t1), std::forward<T2>(t2));
    }
};
```

It is important to highlight the following:

— the operation identity value is a combination of input range value type and the operation itself. That's why `operation_identity` is a variable template for compile-time known values. For `float` we likely want an accumulator variable to have a `float` within an algorithm implementation, while for `int` the choice is likely to be different.

— For `std::plus` the operation identity could be just `IdentityType{}` (for transparent `plus`) without `0` inside and without extra constraints to be a fundamental type. While it enables some use cases, like reductions over `std::string`, it could also bring some harm if the default-constructed value of a value type is not an operation identity, which will lead to surprises at run-time. Also, we cannot do `IdentityType{}` for `std::multiply`; we have to pass `1` as a constructor argument. Based on that, for now we decided that we will provide more restrictive API, which is actually has more predictable behavior and also consistent across different binary operations.

— Users can always override identity value of any operation by writing their own callable wrapper or by using a `binary_operation` convenience wrapper shown below. In fact, we believe that `binary_operation` wrapper should be sufficient in 99.9% use cases when identity should be overridden.

```
template<class BinaryOp, class Identity>
struct binary_operation {
  decltype(auto) operator()(auto&& a, auto&& b) const
  {
    return op(std::forward<decltype(a)>(a), std::forward<decltype(b)>(b));
  }

  [[no_unique_address]] BinaryOp op;
  [[no_unique_address]] Identity operation_identity;
};
```

Eventually, `binary_operation` should also distinguish between compile-time and run-time identity values providing a variable template for the former. The implementation experience is work-in-progress.

A call to an API would look like this:

```
// With the proposed changes the identity is fetch from `std::plus`
        automagically
ranges::exclusive_scan(in, out, initial_value, std::plus{});

// With the proposed changes the identity is fetch from `std::binary_oper-
        ation` automagically
p3732::exclusive_scan(in, out, initial_value,
  p3732::binary_operation{
    [](auto x, auto y) { return x + y; },
     0 // This is operation identity value that can be overriden here, if
        necessary
  }
);
```

A more detailed design sketch to what's written above can be found here. This design is, essentially, a refinement of what
Binary operation wrapper that can hold identity too section describes.

For more information about why we propose to have operation identity and what options we considered, please look at Specifying an identity for reductions and scans.

§ 4 Scope rationale and design

[P3179R9], "C++ Parallel Range Algorithms," is accepted to C++ working draft for C++26. [P3179R9] explicitly defers adding `ranges` versions of the numeric algorithms. This proposal does that. As such, we focus on the 11 algorithms in [numeric.ops].

— `iota`

— `accumulate`

— `inner_product`

— `partial_sum`

— `adjacent_difference`

— `reduce`

— `inclusive_scan`

— `exclusive_scan`

— `transform_reduce`

— `transform_inclusive_scan`

— `transform_exclusive_scan`

We don't have to add ranges versions of all these algorithms. Several already have a ranges version in C++23, possibly with a different name. Some others could be omitted because they have straightforward replacements using existing views and other `ranges` algorithms. We carefully read the two proposals [P2214R2], "A Plan for C++23 Ranges," and [P2760R1], "A Plan for C++26 Ranges," in order to inform our algorithm selections. In some cases that we will explain below, usability and performance concerns led us to disagree with their conclusions.

Additionally, we consider variations of reduction algorithms that are not present in the C++ standard.

§ 4.1 `*_reduce` and `*_scan` algorithms

§ 4.1.1 Summary

We propose

— providing both unary and binary `ranges::transform_reduce` as well as `ranges::reduce`,

— providing `ranges::transform_{in,ex}clusive_scan` as well as `ranges::{in,ex}clusive_scan`, and

— *not* providing projections for any of these algorithms.

§ 4.1.2 Do we want `transform_*` algorithms and/or projections?

We start with two questions.

1. Should the existing C++17 algorithms `transform_reduce`, `transform_inclusive_scan`, and `transform_exclusive_scan` have ranges versions, or does it suffice to have ranges versions of `reduce`, `inclusive_scan`, and `exclusive_scan`?
2. Should ranges versions of `reduce`, `inclusive_scan`, and `exclusive_scan` take optional projections, just like `ranges::for_each` and other `ranges` algorithms do?

We use words like "should" because the ranges library doesn't actually *need* `transform_*` algorithms or projections for functional completeness. These questions are about usability and optimization, including the way that certain kinds of ranges constructs can hinder parallelization on different kinds of hardware.

§ 4.1.3 Unary transforms, projections, and `transform_view` are functionally equivalent

The above two questions are related, since a projection can have the same effect as a `transform_*` function. This aligns with Section 13.2 of N4128, which explains why `ranges` algorithms take optional projections "everywhere it makes sense."

> Wherever appropriate, algorithms should optionally take *`INVOKE`*-able *projections* that are applied to each element in the input sequence(s). This, in effect, allows users to trivially transform each input sequence for the sake of that single algorithm invocation.

Projecting the input of `reduce` has the same effect as unary `transform_reduce`. Here is an example, in which `get_element` is a customization point object like the one proposed in [P2769R3], such that `get_element<k>` gets the `k`-th element of an object that participates in the tuple or structured binding protocol.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
constexpr int init = 3;
auto result_proj =
  std::ranges::reduce(v1, init, std::plus{}, get_element<0>{});
assert(result_proj == 26);
auto result_xform =
  std::ranges::transform_reduce(v1, init, std::plus{}, get_element<0>{});
assert(result_xform == 26);
```

Even without projections, the `transform_*` algorithms can be replaced by a combination of `transform_view` and the non-`transform` algorithm.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
constexpr int init = 3;
auto result_tv = std::ranges::reduce(
  std::views::transform(v1, get_element<0>{}), init, std::plus{});
assert(result_tv == 26);
```

This applies to scan algorithms as well. [P2214R2] points out that `ranges::transform_inclusive_scan(r, o, f, g)` can be rewritten as `ranges::inclusive_scan(r | views::transform(g), o, f)`. The latter formulation saves users from needing to remember which of `f` and `g` is the transform (unary) operation, and which is the binary operation. Making the ranges version of the algorithm take an optional projection would be exactly equivalent to adding a `transform_*` version that does not take a projection: e.g., `ranges::inclusive_scan(r, o, f, g)` with `g` as the projection would do exactly the same thing as `ranges::transform_inclusive_scan(r, o, f, g)` with `g` as the transform operation.

§ 4.1.4 Binary `transform_reduce` is functionally equivalent to `reduce` and `zip_transform_view`

The binary variant of `transform_reduce` is different. Unlike `reduce` and most other numeric algorithms, it takes two input sequences and applies a binary function to the pairs of elements from both sequences. Projections, being unary functions, cannot replace the binary transform function of the algorithm. Likewise, `transform_view` by itself cannot replace the binary transform function unless it is combined with `zip_view` and operates on tuples of elements. `zip_transform_view` is a convenient way to express this combination; applying `reduce` to `zip_transform_view` gives the necessary result (code examples are shown below).

§ 4.1.5 Study `ranges::transform` for design hints

Questions about transforms and projections suggest studying `ranges::transform` for design hints. This leads us to two more questions.

1. If transforms and projections are equivalent, then why does `std::ranges::transform` take an optional projection?
2. If binary transform is equivalent to unary transform of a `zip_transform_view`, then why does binary `std::ranges::transform` exist?

§ 4.1.5.1 Binary transform

It can help to look at examples. The code below shows the same binary transform computation done in two different ways: without projections and with projections.

`transform` without and with projections

Without projections

```
struct foo {};
std::vector<std::tuple<int, foo,
        std::string>> v1{
  {5, {}, "five"}, {7, {},
        "seven"}, {11, {},
        "eleven"}};
std::vector<std::pair<int,
        std::string>> v2{
  {13, "thirteen"}, {17,
        "seventeen"}, {19,
        "nineteen"}};
std::vector<int>
        out(std::from_range,
        std::views::repeat(0,
        3));;
std::vector<int> expected{65,
        119, 209};

// Without projections: Big,
        opaque lambda
std::ranges::transform(v1, v2,
        out.begin(),
  [] (auto x, auto y) { return
        get<0>(x) * get<0>(y);
        });
assert(out == expected);
```

With projections

```
struct foo {};
std::vector<std::tuple<int, foo,
        std::string>> v1{
  {5, {}, "five"}, {7, {},
        "seven"}, {11, {},
        "eleven"}};
std::vector<std::pair<int,
        std::string>> v2{
  {13, "thirteen"}, {17,
        "seventeen"}, {19,
        "nineteen"}};
std::vector<int>
        out(std::from_range,
        std::views::repeat(0,
        3));;
std::vector<int> expected{65,
        119, 209};

// With projections: More
        readable
std::ranges::transform(v1, v2,
        out.begin(),
  std::multiplies{},
        get_element<0>{},
        get_element<0>{});
assert(out2 == expected);
```

The code without projections uses a single big lambda to express the binary operation. Users have to read the big lambda to see what it does. So does the compiler, which can hinder optimization if it's not good at inlining. In contrast, the version with projections lets users read out loud what it does. It also separates the "selection" or "query" part of the transform from the "arithmetic" or "computation" part. The power of the ranges abstraction is that users can factor computation on a range from the logic to iterate over that range. It's natural to extend this separation to selection logic as well.

§ 4.1.5.2 Unary transform

In the unary `transform` case, it's harder to avoid using a lambda. Most of the named C++ Standard Library arithmetic function objects are binary. Currying them into unary functions in C++ requires either making a lambda (which defeats the purpose somewhat) or using something like `std::bind_front` (which is verbose). On the other hand, using a projection still has the benefit of separating the "selection" part of the transform from the "computation" part.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
```

```
std::vector<int> out(std::from_range, std::views::repeat(0, 3));
std::vector<int> expected{6, 8, 12};

// Unary transform without projection
std::ranges::transform(v1, out.begin(), [] (auto x) { return get<0>(x) +
        1; });
assert(out == expected);

// Unary transform with projection
std::ranges::transform(v1, out.begin(), [] (auto x) { return x + 1; },
        get_element<0>{});
assert(out == expected);

// Unary transform with projection and "curried" plus
std::ranges::transform(v1, out.begin(), std::bind_front(std::plus{}, 1),
        get_element<0>{});
assert(out == expected);
```

§ 4.1.6 reduce: transforms and projections

We return to the `reduce` examples we showed above, but this time, we focus on their readability.

§ 4.1.6.1 Unary `transform_reduce`

A `ranges::reduce` that takes a projection is functionally equivalent to unary `transform_reduce` without a projection. If `ranges` algorithms take projections whenever possible, then the name `transform_reduce` is redundant here. Readers should know that any extra function argument of a ranges algorithm is most likely a projection. Either way – `reduce` with projection, or unary `transform_reduce` – is straightforward to read, and separates selection (`get_element<0>`) from computation (`std::plus`).

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
constexpr int init = 3;

// reduce with projection get_element<0>
auto result_proj =
  std::ranges::reduce(v1, init, std::plus{}, get_element<0>{});
assert(result_proj == 26);

// transform_reduce with unary transform get_element<0>
auto result_xform =
  std::ranges::transform_reduce(v1, init, std::plus{}, get_element<0>{});
assert(result_xform == 26);

// reduce with transform_view (no projection)
auto result_xv = std::ranges::reduce(
  std::views::transform(v1, get_element<0>{}), init, std::plus{});
assert(result_xv == 26);
```

On the other hand, `ranges` algorithms take projections whenever possible, and `std::ranges::transform` takes a projection. Why can't `transform_reduce` take a projection? For unary `transform_reduce`, this arguably makes the order of operations less clear. The projection happens first, but most users would have to think about that. A lambda or named function would improve readability.

```
struct bar {
  std::string s;
  int i;
};
std::vector<std::tuple<int, std::string, bar>> v{
  { 5,   "five", {"x", 13}},
  { 7,  "seven", {"y", 17}},
  {11, "eleven", {"z", 19}}};
constexpr int init = 3;

// first get bar, then get bar::i
auto result_proj = std::ranges::transform_reduce(
  v, init, std::plus{}, get_element<1>{}, get_element<2>{});
assert(result_proj == 52);

// first get bar, then get bar::i
auto getter = [] (auto t) {
   return get_element<1>{}(get_element<2>{}(t)); // imagine that get_ele-
        ment works for structs
};
auto result_no_proj = std::ranges::transform_reduce(
  v, init, std::plus{}, getter);
assert(result_no_proj == 52);
```

§ 4.1.6.2 Binary `transform_reduce`

As we explained above, expressing the functionality of binary `transform_reduce` using only `reduce` requires `zip_transform_view` or something like it. This makes the `reduce`-only version more verbose. Users may also find it troublesome that `zip_view` and `zip_transform_view` are not pipeable: there is no `{v1, v2} | views::zip` syntax, for example. On the other hand, it's a toss-up which version is easier to understand. Users either need to learn what `zip_transform_view` does, or they need to learn about `transform_reduce` and know which of the two function arguments does what.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
std::vector<std::pair<std::string, int>> v2{
  {"thirteen", 13}, {"seventeen", 17}, {"nineteen", 19}};
constexpr int init = 3;

// reduce with zip_transform_view
auto result_bztv = std::ranges::reduce(
  std::views::zip_transform(std::multiplies{},
    std::views::transform(v1, get_element<0>{}),
    std::views::transform(v2, get_element<1>{})),
  init, std::plus{});
```

```
assert(result_bztv == 396);

// binary transform_reduce
auto result_no_proj = std::ranges::transform_reduce(
  std::views::transform(v1, get_element<0>{}),
  std::views::transform(v2, get_element<1>{}),
  init, std::plus{}, std::multiplies{});
assert(result_no_proj == 396);
```

C++17 binary `transform_reduce` does not take projections. Instead, it takes a binary transform function, that combines elements from the two input ranges into a single value. The algorithm then reduces these values using the binary reduce function and the initial value. It's perhaps misleading that this binary function is called a "transform"; it's really a kind of "inner" reduction on corresponding elements of the two input ranges.

One can imagine a ranges analog of C++17 binary `transform_reduce` that takes two projection functions, as in the example below. The result has four consecutive function arguments in a row, which is more than for any other algorithm in the Standard Library. Without projections, users need to resort to `transform_view`, but this more verbose syntax makes it more clear which functions do what.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
std::vector<std::pair<std::string, int>> v2{
  {"thirteen", 13}, {"seventeen", 17}, {"nineteen", 19}};
constexpr int init = 3;

// With projections: 4 functions in a row
auto result_proj = std::ranges::transform_reduce(v1, v2, init,
  std::plus{}, std::multiplies{}, get_element<0>{}, get_element<1>{});
assert(result_proj == 396);

// Without projections: more clear where get_element<k> happens
auto result_no_proj = std::ranges::transform_reduce(
  std::views::transform(v1, get_element<0>{}),
  std::views::transform(v2, get_element<1>{}),
  init, std::plus{}, std::multiplies{});
assert(result_no_proj == 396);
```

§ 4.1.7 Mixed guidance from the current ranges library

The current ranges library offers only mixed guidance for deciding whether `*reduce` algorithms should take projections.

The various `fold_*` algorithms take no projections. Section 4.6 of [P2322R6] explains that the `fold_left_first` algorithm does not take a projection in order to avoid an extra copy of the leftmost value, that would be required in order to support projections with a range whose iterators yield proxy reference types like `tuple<T&>` (as `views::zip` does). [P2322R6] clarifies that `fold_left_first`, `fold_right_last`, and `fold_left_first_with_iter` all have this issue. However, the remaining two `fold_*` algorithms

`fold_left` and `fold_right` do not. This is because they never need to materialize an input value; they can just project each element at iterator `iter` via `invoke(proj, *iter)`, and feed that directly into the binary operation. The author of [P2322R6] has elected to omit projections for all five `fold_*` algorithms, so that they have a consistent interface.

A ranges version of `reduce` does not have `fold_left_first`'s design issue. C++17 algorithms in the `reduce` family can copy results as much as they like, so that would be less of a concern here. However, if we ever wanted a `ranges::reduce_first` algorithm, then the consistency argument would arise.

4.1.8 `*transform_view` not always trivially copyable even when function object is

Use of `transform_view` and `zip_transform_view` can make it harder for implementations to parallelize `ranges` algorithms. The problem is that both views might not necessarily be trivially copyable, even if their function object is. If a range isn't trivially copyable, then the implementation must do more work beyond just a `memcpy` or equivalent in order to get copies of the range to different parallel execution units.

Here is an example (available on Compiler Explorer as well).

```
#include <ranges>
#include <type_traits>
#include <vector>

// Function object type that acts just like f2 below.
struct F3 {
  int operator() (int x) const {
    return x + y;
  }
  int y = 1;
};

int main() {
  std::vector v{1, 2, 3};

  // operator= is defaulted; lambda type is trivially copyable
  auto f1 = [] (auto x) {
    return x + 1;
  };
  static_assert(std::is_trivially_copyable_v<decltype(f1)>);

  // Capture means that lambda's operator= is deleted,
  // but lambda type is still trivially copyable
  auto f2 = [y = 1] (auto x) {
    return x + y;
  };
  static_assert(std::is_trivially_copyable_v<decltype(f2)>);

  // decltype(view1) is trivially copyable
  auto view1 = v | std::views::transform(f1);
  static_assert(std::is_trivially_copyable_v<decltype(view1)>);
```

```
  // decltype(view2) is NOT trivially copyable, even though f2 is
  auto view2 = v | std::views::transform(f2);
  static_assert(!std::is_trivially_copyable_v<decltype(view2)>);

  // view3 is trivally copyable, though it behaves just like view2.
  F3 f3{};
  auto view3 = v | std::views::transform(f3);
  static_assert(std::is_trivially_copyable_v<decltype(view3)>);

  return 0;
}
```

Both lambdas `f1` and `f2` are trivially copyable, but `std::views::transform(f2)` is *not* trivally copyable. The wording for both `transform_view` and `zip_transform_view` expresses the input function object of type `F` as stored in an exposition-only *`movable-box<F>`* member. `f2` has a capture that gives it a `=deleted` copy assignment operator. Nevertheless, `f2` is still trivially copyable, because each of its default copy and move operations is either trivial or deleted, and its destructor is trivial and not deleted.

The problem is *`movable-box`*. As [range.move.wrap] 1.3 explains, since `copyable<decltype(f2)>` is not modeled, *`movable-box`*`<decltype(f2)>` provides a nontrivial, not deleted copy assignment operator. This makes *`movable-box`*`<decltype(f2)>`, and therefore `transform_view` and `zip_transform_view`, not trivially copyable.

This feels like a wording bug. `f2` is a struct with one member, an `int`, and a call operator. Why can't I `memcpy` `views::transform(f2)` wherever I need it to go? Even worse, `f3` is a struct just like `f2`, yet `views::transform(f3)` is trivially copyable.

Implementations can work around this in different ways. For example, an implementation of `std::ranges::reduce` could have a specialization for the range being `zip_transform_view<F, V1, V2>` that reaches inside the `zip_transform_view`, pulls out the function object and views, and calls the equivalent of binary `transform_reduce` with them. However, the ranges library generally wasn't designed to make such transformations easy to implement in portable C++. Views generally don't expose their members – an issue that hinders all kinds of optimizations. (For instance, it should be possible for compilers to transform `cartesian_product_view` of bounded `iota_view` into OpenACC or OpenMP multidimensional nested loops for easier optimization, but `cartesian_product_view` does not have a standard way to get at its member view(s).) As a result, an approach based on specializing algorithms for specific view types means that implementations cannot straightforwardly depend on a third-party ranges implementation for their views. Parallel algorithm implementers generally prefer to minimize coupling of actual parallel algorithms with Standard Library features that don't directly relate to parallel execution.

§ 4.1.9 Review

Let's review what we learned from the above discussion.

— Projections improve readability of `ranges::transform`.

— Projections expose optimization potential, by separating the selection part of an algorithm from the computation part.

— None of the existing `ranges::fold_*` algorithms (the closest things the Standard Library currently has to `ranges::reduce`) take projections.

— `reduce` with a projection and unary `transform_reduce` without a projection have the same functionality, without much usability or implementation difference. Ditto for `{in,ex}clusive_scan` with a projection and `transform_{in,ex}clusive_scan` without.

— Expressing binary `transform_reduce` using only `reduce` requires `zip_transform_view` *always*, even if the two input ranges are contiguous ranges of `int`. This hinders readability and potentially also performance.

— A ranges version of binary `transform_reduce` that takes projections is harder to use and read than a version without projections. However, a version without projections would need `transform_view` in order to offer the same functionality. This potentially hinders performance.

§ 4.1.10 Conclusions

We propose

— providing both unary and binary `ranges::transform_reduce` as well as `ranges::reduce`,

— providing `ranges::transform_{in,ex}clusive_scan` as well as `ranges::{in,ex}clusive_scan`, and

— *not* providing projections for any of these algorithms.

We conclude this based on a chain of reasoning, starting with binary `transform_reduce`.

1. We want binary `transform_reduce` for usability and performance reasons. (The "transform" of a binary `transform_reduce` is *not* the same thing as a projection.)
2. It's inconsistent to have binary `transform_reduce` without unary `transform_reduce`.
3. Projections tend to hinder usability of both unary and binary `transform_reduce`. If we have unary `transform_reduce`, we don't need `reduce` with a projection.
4. We already have `fold_*` (effectively special cases of `reduce`) without projections, even though some of the `fold_*` algorithms *could* have had projections.
5. If we have other `*reduce` algorithms without projections as well, then the most consistent thing would be for *no* reduction algorithms to have projections.
6. It's more consistent for the various `*scan` algorithms to look and act like their `*reduce` counterparts, so we provide `ranges::transform_{in,ex}clusive_scan` as

well as `ranges::{in,ex}clusive_scan`, and do not provide projections for any of them.

§ 4.2 Convenience wrappers to replace some algorithms

§ 4.2.1 `accumulate`

The `accumulate` algorithm performs operations sequentially. Users who want that left-to-right sequential behavior can call C++23's `fold_left`. For users who are not concerned about the order of operations and who want `accumulate`'s default binary operation, we propose parallel and non-parallel convenience wrappers `ranges::sum`.

§ 4.2.2 `inner_product`

The `inner_product` algorithm performs operations sequentially. Users who want that left-to-right sequential behavior can call `fold_left`. Note that [P2214R2] argues specifically against adding a ranges analog of `inner_product`, because it is less fundamental than other algorithms.

For users who are not concerned about the order of operations and who want the default binary operations used by `inner_product`, we propose parallel and non-parallel convenience wrappers `ranges::dot`. We call them `dot` and not `inner_product` because inner products are mathematically more general. We specifically mean not just any inner product, but the inner product that is the dot product. Calling them `dot` has the added benefit that they represent the same mathematical computation as `std::linalg::dot`.

§ 4.3 `reduce_into` and `transform_reduce_into`

We propose new parallel and non-parallel algorithms `reduce_into` and `transform_reduce_into`. These work like `reduce` and `transform_reduce`, except that instead of returning the reduction result by value, they write it to the first element of an output range. We include both unary and binary versions of `transform_reduce_into`. We also provide convenience wrappers `sum_into`, `product_into`, and `dot_into` that are the "`_into`" analogues of `sum`, `product`, and `dot`.

§ 4.3.1 Justification

The `reduce_into` algorithm has precedent in the Thrust library. Its performance advantage is that the algorithm can write its result directly to special memory associated with parallel execution, such as accelerator memory or a NUMA (Non-Uniform Memory Access) domain where the algorithm's threads run.

§ 4.3.2 Provide both parallel and non-parallel versions of these algorithms

C++17 offers both parallel and non-parallel `reduce`, `transform_reduce`, `inclusive_scan`, and `exclusive_scan`. The main benefit of the non-parallel versions is that they per-

mit reordering terms in the reduction or sum. For example, an implementation of `reduce(x.begin(), x.end(), std::plus{})` for a forward range of `float` is permitted to copy the range into contiguous storage and perform a parallel- and SIMD-accelerated reduction there. We want our non-parallel `ranges` numeric algorithms to have the same implementation freedom.

§ 4.3.3 Output should be a sized forward range, not an iterator

[P3179R9] (parallel ranges algorithms) always specifies output data as sized ranges, instead of as a single iterator. However, in the case of `*reduce_into`, the output consists of only one element. Thus, the interface could represent the output either as a single iterator to that element, or as a range. We propose representing the output as a sized forward range.

There are two parts to this.

1. *Sized*: we define this as `sized_range`, and say that the algorithm only writes to it if it has nonzero `ranges::size(r)`

2. *Forward range*: in the sense of `forward_range`

We propose making the output a sized range instead of just an iterator because this lets the `*reduce_into` algorithms simply do nothing if the output range has zero size. This would make their behavior consistent with [P3179R9]'s parallel ranges algorithms that were adopted into the Working Draft for C++26.

Unlike other algorithms in [P3179R9], the `*reduce_into` algorithms don't need to know the size of the output range; they just need to know that it has at least one element. Ranges provides three different ways to say that a range `r` has at least one element.

a. The range is a `sized_range` (meaning that `ranges::size(r)` has constant complexity) and `ranges::size(r)` is nonzero;

b. `ranges::empty(r)` is `false`; or,

c. iterator comparison: `ranges::begin(r) != ranges::end(r)`.

We choose Option (a), `sized_range`, because

— it's consistent with [P3179R9]'s output ranges;

— the Standard currently has no concept to express algorithmic complexity constraints on `ranges::empty(r)` ([range.prim.empty]), while `ranges::size(r)` on a `sized_range` always has constant complexity; and

— `sized_range` permits evaluation of `ranges::size(r)` before `ranges::begin(r)` without invalidating the range, even if the range is not a `forward_range`. This would enable future proposals that generalize the output to be a `sized_range` that is not a `forward_range`.

We propose making the output a `forward_range` because the intention of these algorithms is that once they return, users read from the output range. If the algorithm itself invalidates the range by writing to it, then users can't use the output range to get back the result. This requirement applies to both the parallel and the non-parallel algorithms. Specifically for the parallel algorithms, the output range must be copyable. The Standard does not currently have an iterator category to express "single-pass but copyable." This, again, would limit the iterator category to be at least forward.

§ 4.3.4 Use case comparing range and iterator interface options

The motivating use case for `*reduce_into` is that both input and output live in special memory associated with parallel execution. Users of accelerators may want to avoid implicitly reallocating data structures like `std::vector`, and instead make all allocations explicit, like this.

```
// Allocate num_bytes bytes of special memory
extern void* accelerator_malloc(std::size_t num_bytes);
// Free an allocation created by accelerator_malloc
extern void accelerator_free(void* ptr);

// unique_ptr deleter for special memory
template<class T>
struct accelerator_deleter {
  void operator() (T* ptr) const {
    accelerator_free(ptr);
  }
};

// Dynamic allocation of special memory for an array
template<class T, std::size_t Extent = std::dynamic_extent>
class accelerator_array {
public:
  accelerator_array(std::size_t num_elements) :
    num_elements_(num_elements),
    alloc_((float*) accelerator_malloc(num_elements * sizeof(T)), {})
  {}

  std::span<T, Extent> get_span() const {
    if constexpr (Extent == std::dynamic_extent) {
      return {alloc_.get(), num_elements_}
    }
    else {
      return {alloc_.get()};
    }
  }

private:
  [[no_unique_address]] std::extents<std::size_t, Extent> num_elements_;
  std::unique_ptr<T[], accelerator_deleter<T>> alloc_;
};

// Dynamic allocation of special memory for a single value
template<class T>
```

```
class accelerator_value {
public:
  accelerator_value() :
    alloc_((float*) accelerator_malloc(sizeof(T)), {})
  {}

  T* get() const {
    return alloc_.get();
  }

private:
  std::unique_ptr<T, accelerator_deleter<T>> alloc_;
};

// Fill x with some values
extern void user_fill_span(std::span<float> x);
```

If `reduce_into` takes a range for the output, users would use it like this. Note that the output value needs to be a range. We do that here by making it a size-1 `span`, but there are other options.

```
// Create input range, reduce over input into output,
// and return output allocation.
accelerator_array<float, 1>
user_fill_and_reduce(std::size_t num_elements) {
  accelerator_array<float> input(num_elements);
  user_fill_span(input.get_span());
  accelerator_value<float> output;
  std::ranges::reduce_into(std::execution::par,
    input.get_span(),
    std::span<float, 1>(output.get())
    /*, other arguments */);
  return std::move(output);
}
```

If `reduce_into` takes an iterator for the output, users would use it like this.

```
// Create input range, reduce over input into output,
// and return output allocation.
accelerator_value<float>
user_fill_and_reduce(std::size_t num_elements) {
  accelerator_array<float> input(num_elements);
  user_fill_span(input.get_span());
  accelerator_value<float> output;
  std::ranges::reduce_into(std::execution::par,
    input.get_span(),
    output.get() // pointer is an iterator
    /*, other arguments */);
  return std::move(output);
}
```

The above examples represent the intended and likely most common use case for `*reduce_into`. If users already have a `float result;` on the stack, they don't need to fuss

with pointers or `span`; they can just call `reduce` and assign to `result`. Thus, users probably won't be writing code like this.

```
std::vector<float> input_range{3.0f, 5.0f, 7.0f};
float output_value{};
ranges::reduce_into(std::execution::par,
  input_range,
  span<float, 1>{&output_value}
  /*, other arguments */);
assert(out_value == out[0]);
```

§ 4.3.5 Add `sum_into`, `product_into`, and `dot_into`

We provide convenience wrappers `ranges::sum` and `ranges::product` for special cases of `reduce` with addition resp. multiplication, and `ranges::dot` for the special case of binary `transform_reduce` with transform `multiplies{}` and reduction `plus{}`. As a result, we also need to provide `_into` versions: `sum_into`, `product_into`, and `dot_into`. Otherwise, users who want reductions for these special cases would have to write them by hand and call `reduce_into` or `transform_reduce_into`.

§ 4.3.6 Conclusions

1. Include both parallel and non-parallel versions of `reduce_into` and `transform_reduce_into`.

2. Represent the output as a `sized_range` + `forward_range`.

3. Include both parallel and non-parallel versions of `sum_into`, `product_into`, and `dot_into`.

§ 4.4 Other existing algorithms can be replaced with views

§ 4.4.1 `iota`

C++20 has `iota_view`, the view version of `iota`. One can replace the `iota` algorithm with `iota_view` and `ranges::copy`. In fact, one could argue that `iota_view` is the perfect use case for a view: instead of storing the entire range, users can represent it compactly with two integers. There also should be no optimization concerns with parallel algorithms over an `iota_view`. For example, the Standard specifies `iota_view` in a way that does not hinder it from being trivially copyable, as long as its input types are. The iterator type of `iota_view` is a random access iterator for reasonable lower bound types (e.g., integers).

However, `ranges::iota` algorithm was added since C++23, later than `iota_view`. For the sake of completeness we might want to add a parallel variation of it as well. It's only going to give a syntactic advantage: if users already have `ranges::iota` in their code, parallelizing it would be as simple as adding an execution policy (assuming the iterator/range categories are satisfied).

We do not propose parallel `ranges::iota` for now. We are seeking for SG9 (Ranges Study Group) feedback.

§ 4.4.2 `adjacent_difference`

The `adjacent_difference` algorithm can be replaced with a combination of `adjacent_transform_view` (which was adopted in C++23) and `ranges::copy`. We argue elsewhere in this proposal that views (such as `adjacent_transform_view`) that use a *`movable-box<F>`* member to represent a function object may have performance issues, due to *`movable-box<F>`* being not trivially copyable even for some cases where `F` is trivially copyable. On the other hand, the existing `adjacent_difference` with the default binary operation (subtraction) could be covered with the trivially copyable `std::minus` function object.

In our experience, adjacent differences or their generalization are often used in combination with other ranges. For example, finite-difference methods (such as Runge-Kutta schemes) for solving time-dependent differential equations may need to add together multiple ranges, each of which is an adjacent difference possibly composed with other functions. If users want to express that as a one-pass algorithm, they might need to combine more than two input ranges, possibly using a combination of `transform_views` and `adjacent_transform_views`. This ultimately would be hard to express as a single "`ranges::adjacent_transform`" algorithm invocation. Furthermore, `ranges::adjacent_transform` is necessarily single-dimensional. It could not be used straightforwardly for finite-difference methods for solving partial differential equations, for example. All this makes an `adjacent_transform` algorithm a lower-priority task.

We do not propose `adjacent_transform` for the reasons described above.

§ 4.4.3 `partial_sum`

The `partial_sum` algorithm combines elements sequentially, from left to right. It behaves like an order-constrained version of `inclusive_scan`.

Our proposal focuses on algorithms that permit reordering binary operations. For users who want an order-constrained partial sum, [P3351R2], "`views::scan`," proposes a view with the same left-to-right behavior. This paper is currently in SG9 (Ranges Study Group) review.

Users of `partial_sum` who are not concerned about the order of operations can call the `inclusive_scan` algorithm (proposed here) instead. We considered adding a convenience wrapper for the same special case of an inclusive prefix plus-scan that `partial_sum` supports. However, names like `partial_sum` or `prefix_sum` would obscure whether this is an inclusive or exclusive scan. Also, the existing `partial_sum` algorithm operates left-to-right. A new algorithm with the same name and almost the same interface, but with a different order of operations, could be misleading. We think it's not a very convenient convenience wrapper if users have to look up its behavior every time they use it.

If WG21 did want a convenience wrapper, one option would be to give this common use case a longer but more explicit name, like `inclusive_sum_scan`.

§ 4.5 We don't propose "the lost algorithm" (noncommutative parallel reduce)

The Standard lacks an analog of `reduce` that can assume associativity but not commutativity of binary operations. One author of this proposal refers to this as "the lost algorithm." (Please refer to Episode 25 of "ASDP: The Podcast".) We do not propose this algorithm, but we would welcome a separate proposal to do so.

The current numeric algorithms express a variety of permissions to reorder binary operations.

— `accumulate` and `partial_sum` both precisely specify the order of binary operations as sequential, from left to right. This works even if the binary operation is neither associative nor commutative.

— The various `*_scan` algorithms can reorder binary operations as if they are associative (that is, it is allowed to replace `a + (b + c)` with `(a + b) + c`), but not as if they are commutative (it is prohibited to replace `a + b` with `b + a`).

— `reduce` can reorder binary operations as if they are both associative and commutative.

What's missing here is a parallel analog of `reduce` with the assumptions of `*_scan`, that is, a reduction that can assume associativity but not commutativity of binary operations. Parallel reduction operations with these assumptions exist in other programming models. For example, MPI (the Message Passing Interface for distributed-memory parallel communication) has a function `MPI_Create_op` for defining custom reduction operators from a user's function. `MPI_Create_op` has a parameter that specifies whether MPI may assume that the user's function is commutative.

Users could get that parallel algorithm by calling `*_scan` with an extra output sequence, and using only the last element. However, this requires extra storage.

A concepts-based approach like [P1813R0]'s could permit specializing `reduce` on whether the user asserts that the binary operation is commutative. [P1813R0] does not attempt to do this; it merely specializes `reduce` on whether the associative and commutative operation has a two-sided identity element. Furthermore, [P1813R0] does not offer a way for users to assert that an operation is associative or commutative, because the `magma` (non-associative) and `semigroup` (associative) concepts do not differ syntactically. One could imagine a refinement of this design that includes a trait for users to specialize on the type of their binary operation, say `is_commutative<BinaryOp>`. This would be analogous to the `two_sided_identity` trait in [P1813R0] that lets users declare that their set forms a monoid, a refinement of `semigroup` with a two-sided identity element.

This proposal leaves the described algorithm out of scope. We think the right way would be to propose a new algorithm with a distinct name. A reasonable choice of name would be `fold` (just `fold` by itself, not `fold_left` or `fold_right`).

§ 4.6 We don't propose `reduce_with_iter`

A hypothetical `reduce_with_iter` algorithm would look like `fold_left_with_iter`, but would permit reordering of binary operations. It would return both an iterator to one past the last input element, and the computed value. The only reason for a reduction to return an iterator would be if the input range is single-pass. However, users who have a single-pass input range really should be using one of the `fold*` algorithms instead of `reduce*`. As a result, we do not propose the analogous `reduce_with_iter` here.

Note that the previous paragraph effectively argues for `*reduce` to require at least forward ranges.

Just like `fold_left`, the `reduce` algorithm should return just the computed value. Section 4.4 of [P2322R6] argues that this makes it easier to use, and improves consistency with other `ranges` algorithms like `ranges::count` and `ranges::any_of`. It is also consistent with [P3179R9]. Furthermore, even if a `reduce_with_iter` algorithm were to exist, `reduce` should not be specified in terms of it. This is for performance reasons, as Section 4.4 of [P2322R6] elaborates for `fold_left` and `fold_left_with_iter`.

§ 4.7 We do not propose `reduce_first` and we do not think it is needed

Section 5.1 of [P2760R1] asks whether the Standard Library should have a "`reduce_first`" algorithm. Analogously to `fold_left_first`, `reduce_first` would use the first element of the range as the initial value of the reduction operation. Users might want this algorithm in cases when

1. there is no reasonable initial value that does not depend on the elements of the range, *and*
2. there is an extra run-time cost for users to read the first element of the range themselves.

We think these requirements together are too esoteric to justify adding a separate algorithm. Furthermore, mitigations exist to work around both requirements.

If an identity for the binary operation exists and the algorithm knows it, it would be the most reasonable initial value independent of the range, because including it in the range of values to reduce would not change the result. Requirement (1) therefore means that the identity for the binary operation does not exist, the user does not know it, or the user has not supplied it to the algorithm. Section Identity may not exist or may be unknown explains that in more details.

The mitigation for lack of a distinct initial value is for users to load the first (or some other) element of the range as the initial value. The element doesn't have to be the first because, Unlike `fold_left_first*` and `fold_right_last`, the `*reduce` algorithms are unordered. The only reason to privilege the first (or last) element is that excluding any other element would not preserve the range's contiguity.

Requirement (2) means that loading the first (or some other) element in the user's code would be slower than the algorithm doing it. For example, for an implementation that executes parallel algorithms using an accelerator such as a GPU, the user's range may live in the accelerator's special memory. Standard C++ requires that this memory be accessible in user's code outside of parallel algorithms, but doesn't require that doing so be fast.

One mitigation for this concern is to use as the initial value a proxy reference to the first element, instead of the first element itself. The proxy reference would defer the load until the algorithm actually uses the value. C++17 `std::reduce` cannot do this because the initial value type *is* the return type. Proxy reference types are not copyable and are certainly not `semiregular`, so they do not make good return types for reductions. Our `reduce` algorithms can use a proxy reference as the initial value because they deduce the return type as the result of the binary operator.

An alternative to `reduce_first` is to make the initial value an optional parameter of `reduce`, allowing an implementation to use any element of the range as the initial value.

§ 4.8 Range categories and return types

We propose the following.

— Our parallel algorithms take sized random access ranges (except for output ranges of "`_into`" algorithms, which are sized forward ranges; see above).

— Our non-parallel algorithms take sized forward ranges.

— Our scans' return type is an alias of `in_out_result`.

— Our reductions just return the reduction value, not `in_value_result` with an input iterator.

— Our `reduction_into` family return type is an alias of `in_out_result`.

[P3179R9] does not aim for perfect consistency with the range categories accepted by existing serial `ranges` algorithms. The algorithms proposed by [P3179R9] differ from serial range algorithms in the following ways.

1. [P3179R9] uses a range, not an iterator, as the output parameter (see Section 2.7).
2. [P3179R9] requires that the ranges be sized (see Section 2.8).
3. [P3179R9] requires random access ranges (see Section 2.6).

Of these differences, (1) and (2) could apply generally to all `ranges` algorithms, so we adopt them for this proposal.

Regarding (1), for arguments in favor of non-parallel algorithms taking a range as output, please refer to [P3490R0], "Justification for ranges as the output of parallel range algorithms." (Despite the title, it has things to say about non-parallel algorithms too.) Taking a range as output would prevent use of existing output-only iterators that do not have a separate sized sentinel type, like `std::back_insert_iterator`. However, all the algorithms we propose require at least forward iterators (see below). [P3490R0] shows that it is possible for both iterator-as-output and range-as-output overloads to coexist, so we follow [P3179R9] by not proposing iterator-as-output algorithms here.

Regarding (2), we make the parallel algorithms proposed here take sized random access ranges, as [P3179R9] does. For consistency, we also propose that the output ranges model `sized_range`. As a result, any parallel algorithms with an output range need to return both an iterator to one past the last element of the output, and an iterator to one past the last element of the input. This tells callers whether there was enough room in the output, and if not, where to start when processing the rest of the input. This includes all the `*{ex,in}clusive_scan` algorithms we propose.

Difference (3) relates to [P3179R9] only proposing parallel algorithms. It would make sense for us to relax this requirement for the non-parallel algorithms we propose. This leaves us with two possibilities:

1. (single-pass) input and output ranges, the most general; or

2. (multipass) forward ranges.

We believe there is no value in `*reduce` and `*_scan` taking single-pass input ranges, because these algorithms can combine the elements of their input range(s) in any order. Suppose that an algorithm had that freedom to rearrange operations, yet was constrained to read the input ranges exactly once, in left-to-right order. The only way such an algorithm could exploit that freedom would be for it to copy the input ranges into temporary storage. Users who want that could just copy the input ranges into contiguous storage themselves.

For this reason, we make the non-parallel algorithms take (multipass) forward ranges, even though this is not consistent with the existing non-parallel `<numeric>` algorithms. If users have single-pass iterators, they should just call one of the `fold_*` algorithms, or use `views::scan` proposed in [P3351R2]. This has the benefit of letting us specify `ranges::reduce` to return just the value. We don't propose a separate `reduce_with_iter` algorithm to return both the value and the one-past-the-input iterator, as we explain in the relevant section.

§ 4.9 Constexpr parallel algorithms?

[P2902R2] proposes to add `constexpr` to the parallel algorithms. [P3179R9] does not object to this; see Section 2.10. We continue the approach of [P3179R9] in not opposing [P2902R2]'s approach, but also not depending on it.

§ 4.10 `ranges::reduce` design

In this section, we focus on `ranges::reduce`'s design. The discussion here applies generally to the other algorithms we propose.

§ 4.10.1 No default binary operation or initial value

Section 5.1 of [P2760R1] states:

> One thing is clear: `ranges::reduce` should *not* take a default binary operation *nor* a default initial [value] parameter. The user needs to supply both.

This motivates the following convenience wrappers:

- `ranges::sum(r)` for `ranges::reduce` with `init = range_value_t<R>()` and `plus{}` as the reduce operation;
- `ranges::product(r)` for `ranges::reduce` with `init = range_value_t<R>(1))` and `multiplies{}` as the reduce operation; and
- `ranges::dot(x, y)` for binary `ranges::transform_reduce` with `init = T()` where `T = decltype(declval<range_value_t<X>>() * declval<range_value_t<Y>>())`, `multiplies{}` is the transform operation, and `plus{}` is the reduce operation.

One argument *for* a default initial value in `std::reduce` is that `int` literals like `0` or `1` do not behave in the expected way with a sequence of `float` or `double`. For `ranges::reduce`, however, making its return value type imitate `ranges::fold_left` instead of `std::reduce` fixes that.

§ 4.10.2 For return type, imitate `ranges::fold_left`, not `std::reduce`

Both `std::reduce` and `std::ranges::fold_left` return the reduction result as a single value. However, they deduce the return type differently. For `ranges::reduce`, we deduce the return type like `std::ranges::fold_left` does, instead of always returning the initial value type `T` like `std::reduce`.

[P2322R6], "`ranges::fold`," added the various `fold_*` `ranges` algorithms to C++23. This proposal explains why `std::ranges::fold_left` may return a different reduction type than `std::reduce` for the same input range, initial value, and binary operation. Consider the following example, adapted from Section 3 of [P2322R6] (Compiler Explorer link).

```
#include <algorithm>
#include <cassert>
```

```cpp
#include <iostream>
#include <numeric>
#include <ranges>
#include <type_traits>
#include <vector>

int main() {
  std::vector<double> v = {0.25, 0.75};
  {
    auto r = std::reduce(v.begin(), v.end(), 1, std::plus());
    static_assert(std::is_same_v<decltype(r), int>);
    assert(r == 1);
  }
  {
    auto r = std::ranges::fold_left(v, 1, std::plus());
    static_assert(std::is_same_v<decltype(r), double>);
    assert(r == 2.0);
  }
  return 0;
}
```

The `std::reduce` part of the example expresses a common user error. `ranges::fold_*` instead returns "the decayed result of invoking the binary operation with `T` (the initial value) and the reference type of the range." For the above example, this likely expresses what the user meant. It also works for other common cases, like proxy reference types with an unambiguous conversion to a common type with the initial value.

It's notable that `reduce`-like `mdspan` algorithms in [linalg] – `dot`, `vector_sum_of_squares`, `vector_two_norm`, `vector_abs_sum`, `matrix_frob_norm`, `matrix_one_norm`, and `matrix_inf_norm` – all have the same return type behavior as C++17 `std::reduce`. However, the authors of [linalg] expect typical users of their library to prefer complete control of the return type, even if it means they have to type `1.0` instead of `1`. These [linalg] algorithms also have more precise wording about precision of intermediate terms in sums when the element types and the initial value are all floating-point types or specializations of `complex`. (See e.g., [linalg.algs.blas1.dot] 7.) For ranges reduction algorithms, we expect a larger audience of users and thus prefer consistency with `fold_*`'s return type.

§ 4.11 Constraining numeric ranges algorithms

In summary,

- We use the same constraints as `fold_left` and `fold_right` to constrain the binary operator of `reduce` and `*_scan`.
- We imitate C++17 parallel algorithms and [linalg] ([P1673R13]) by using *GENERALIZED_NONCOMMUTATIVE_SUM* and *GENERALIZED_SUM* to describe the behavior of `reduce` and `*_scan`.
- Otherwise, we follow the approach of [P3179R9] ("C++ Parallel Range Algorithms").

[P3179R9], which has been voted into the Working Draft for C++26, defines parallel versions of many `ranges` algorithms in the C++ Standard Library. (The "parallel version of an algorithm" is an overload of an algorithm whose first parameter is an execution policy.) That proposal restricts itself to adding parallel versions of existing `ranges` algorithms. [P3179R9] explicitly defers adding overloads to the numeric algorithms in [numeric.ops], because these do not yet have ranges versions. Our proposal fills that gap.

WG21 did not have time to propose ranges-based numeric algorithms with the initial set of `ranges` algorithms in C++20. [P1813R0], "A Concept Design for the Numeric Algorithms," points out the challenge of defining ranges versions of the existing parallel numeric algorithms. What makes this task less straightforward is that the specification of the parallel numeric algorithms permits them to reorder binary operations like addition. This matters because many useful number types do not have associative addition. Lack of associativity is not just a floating-point rounding error issue; one example is saturating integer arithmetic. `Ranges` algorithms are constrained by concepts, but it's not clear even if it's a good idea to define concepts that can express permission to reorder terms in a sum.

C++17 takes the approach of saying that parallel numeric algorithms can reorder the binary operations however they like, but does not say whether any reordering would give the same results as any other reordering. The Standard expresses this through the wording "macros" *GENERALIZED_NONCOMMUTATIVE_SUM* and *GENERALIZED_SUM*. (A wording macro is a parameterized abbreviation for a longer sequence of wording in the Standard. We put "macros" in double quotes because they are not necessarily preprocessor macros. They might not even be implementable as such.) Algorithms become ill-formed, no diagnostic required (IFNDR) if the element types do not define the required operations. [P1813R0] instead defines C++ concepts that represent algebraic structures, all of which involve a set with a closed binary operation. Some of the structures require that the operation be associative and/or commutative. [P1813R0] uses those concepts to constrain the algorithms. This means that the algorithms will not be selected for overload resolution if the element types do not define the required operations. It further means that algorithms could (at least in theory) dispatch based on properties like whether the element type's binary operation is commutative. The concepts include both syntactic and semantic constraints.

WG21 has not expressed a consensus on [P1813R0]'s approach. LEWGI reviewed [P1813R0] at the Belfast meeting in November 2019, but did not forward the proposal and wanted to see it again. Two other proposals express something more like WG21's consensus on constraining the numeric algorithms: [P2214R2], "A Plan for C++23 Ranges," [P1673R13], "A free function linear algebra interface based on the BLAS," which defines `mdspan`-based analogs of the numeric algorithms. Section 5.1.1 of [P2214R2] points out that [P1813R0]'s approach would overconstrain `fold`; [P2214R2] instead suggests just constraining the operation to be binary invocable. This was ultimately the approach taken by the Standard through the exposition-only concepts *`indirectly-binary-left-foldable`* and *`indirectly-binary-right-foldable`*. Section 5.1.2 of [P2214R2] says that `reduce` "calls for the kinds of constraints that [P1813R0] is proposing."

[P1673R13], which was adopted into the Working Draft for C++26 as [linalg], took an entirely different approach for its set of `mdspan`-based numeric algorithms. Section 10.8, "Constraining matrix and vector element types and scalars," explains the argument. Here is a summary.

1. Requirements like associativity are too strict to be useful for practical types. The only number types in the Standard with associative addition are unsigned integers. It's not just a rounding error "epsilon" issue; sums of saturating integers can have infinite error if one assumes associativity.

2. "The algorithm may reorder sums" (which is what we want to say) means something different than "addition on the terms in the sum is associative" (which is not true for many number types of interest). That is, permission for an algorithm to reparenthesize sums is not the same as a concept constraining the terms in the sum.

3. [P1813R0] defines concepts that generalize a mathematical group. These are only useful for describing a single set of numbers, that is, one type. This excludes useful features like mixed precision (e.g., where the result type in `reduce` differs from the range's element type) and types that use expression templates. One could imagine generalizing this to a set of types that have a common type, but this can be too restrictive; Section 5.1.1 of [P2214R2] gives an example involving two types in a fold that do not have a common type.

[P1673R13] says that algorithms have complete freedom to create temporary copies or value-initialized temporary objects, rearrange addends and partial sums arbitrarily, or perform assignments in any order, as long as this would produce the result specified by the algorithm's *Effects* and *Remarks* when operating on elements of a semiring. The `linalg::dot` ([linalg.algs.blas1.dot]) and `linalg::vector_abs_sum` ([linalg.algs.blas1.asum]) algorithms specifically define the returned result(s) in terms of *GENERALIZED_SUM*. Those algorithms do that because they need to constrain the precision of intermediate terms in the sum (so they need to define those terms). In our case, the Standard already uses *GENERALIZED_SUM* and *GENERALIZED_NONCOMMUTATIVE_SUM* to define iterator-based C++17 algorithms like `reduce`, `inclusive_scan`, and `exclusive_scan`. We can just adapt this wording to talk about ranges instead of iterators. This lets us imitate the approach of [P3179R9] in adding ranges overloads.

Our approach combines the syntactic constraints used for the `fold_*` family of algorithms, with the semantic approach of [P1673R13] and the C++17 parallel numeric algorithms. For example, we constrain `reduce`'s binary operation with *`indirectly-binary-foldable`*, which is like saying that it must be both *`indirectly-binary-left-foldable`* and *`indirectly-binary-right-foldable`*. (This expresses that if the binary operation is called with an argument of the initial value's type `T`, then that argument can be in either the first or second position.) We express what `reduce` does using *GENERALIZED_SUM*.

§ 4.12 Enabling list-initialization for proposed algorithms

Our proposal follows the same principles as described in [P2248R8], "Enabling list-initialization for algorithms." We want to enable the use case where users construct a nondefault initial value using curly braces without naming the type.

```
#include <cassert>
#include <numeric>
#include <vector>
#include <functional>

int main() {
  std::vector<double> v = {0.25, 0.75};
  auto r = std::ranges::reduce(v, {1}, std::plus());
  assert(r == 2.0);
}
```

Supporting this use case requires that we add a default template argument to `T init` in the proposed signatures. While [P2248R8] does not propose a default template parameter for `init` in the `<numeric>` header, we want to address this design question from the beginning for the new set of algorithms because `fold_` family already has this feature.

§ 5 Specifying an identity for reductions and scans

§ 5.1 Summary

We propose adding a way for users to *specify an identity value* (or pseudoidentity value; see below) of a binary operation for reductions and scans. This would give parallel implementations a value to use for initializing each execution agent's accumulator.

Parallel reductions and scans don't strictly *require* an identity. Their C++17 versions work fine without it. Not every (mathematically associative and commutative) binary operator has an identity, and figuring out a pseudoidentity may be difficult or impossible. Thus, we propose that the *identity be optional*.

All C++17 reductions and scans have overloads with an initial value parameter. We propose retaining this feature in our ranges reductions and scans. Exclusive scan requires an initial value in order to make mathematical sense, so our ranges `exclusive_scan` and `transform_exclusive_scan` require the initial value parameter. For all other reductions and scans, we propose making the initial value optional, as it is in the C++17 algorithms. For `inclusive_scan` and `transform_inclusive_scan`, the initial value parameter has performance benefits.

The return type of reductions comes from the result of calling the binary operator on the initial value and an element of the range. The identity is optional and is solely an optimization hint. Thus, the identity does not influence our reductions' return type. We only require that

— calling the binary operator with the identity (if provided) and the initial value (in either order) is well formed,

— calling the binary operator with the identity (if provided) and an element of the range (in either order) is well formed, and

— the result of any of these binary operator invocations is assignable to the return type.

Given that we permit both reductions and scans to accept both an initial value and an identity, the interface for providing an identity must help users distinguish it from the initial value. It should also help users see the connection between the identity and the binary operator to which it applies. This matters especially for binary `transform_reduce`, as it takes two binary operators, but the identity would only apply to one of them. We propose

— a *trait for determining whether a binary operator has a known identity value*,

— a *trait for extracting an identity value*, if it exists, from the binary operator, and

— a *wrapper binary operator* that attaches an identity value to the user's binary operator (which may be a lambda or some other type that the user does not control).

Users may want to specify a *compile-time identity value*, that is, a value that is guaranteed to be known at compile time because it results from a `static constexpr` member function of the parameter's type. Examples include the conversion operator of `constant_wrapper` and `integral_constant`. The above interface works with this no differently than with a run-time identity value, because we deduce the return type like `fold_first` does, rather than just making the initial value type the return type like C++17's `std::reduce`.

§ 5.2 Initial value of a reduction or scan

C++17's `reduce`, `transform_reduce`, and `*_scan` algorithms all take an initial value parameter `T init`. This exists for several reasons.

1. For `reduce` and `transform_reduce`, it defines the algorithm's return type, and also the type that the implementation uses for intermediate results.

2. For `*_scan`, it is included in the terms of every partial sum. This can save a pass over the range.

3. For `reduce` and `transform_reduce`, it lets users express a "running reduction" where the whole range is not available all at once and users need to call `reduce` repeatedly.

Both `exclusive_scan` and `transform_exclusive_scan` require an initial value. This is because the first element of the output range is just the initial value. For the other algorithms, the initial value is optional and defaults to `T{}`, a value-initialized `T` value.

§ 5.3 Identity value of a reduction's or scan's binary operator

An *identity value* `id` of a binary operator `bop` is a value such that `bop(x, id)` equals `bop(id, x)` equals `x` for all valid arguments `x` of `bop`. Including an identity value an arbitrary number of times in a reduction does not change the reduction's result.

We say "an" identity value because it need not be unique. For example, if the binary operator is integer addition modulo 7, every multiple of 7 is an identity.

The initial value of a reduction or scan is not necessarily the same as an identity value of the reduction's or scan's binary operator. The identity value can serve as an initial value, but not vice versa. The following example illustrates.

```
std::vector<float> v{5.0f, 7.0f, 11.0f};

// Default initial value is float{}, which is 0.0f.
// It is also the identity for std::plus<>, the default operation.
float result = std::reduce(v.begin(), v.end());
assert(result == 23.0f);

// Initial value happens to be the identity in this case.
result = std::reduce(v.begin(), v.end(), 0.0f);
assert(result == 23.0f);

// Initial value is NOT the identity in this case.
float result_plus_3 = std::reduce(v.begin(), v.end(), 3.0f);
assert(result_plus_3 == 26.0f);

// Including arbitrarily many copies of the identity element
// does not change the reduction result.
std::vector<float> v2{5.0f, 0.0f, 7.0f, 0.0f, 0.0f, 11.0f, 0.0f};
result = std::reduce(v.begin(), v.end());
assert(result == 23.0f);
result = std::reduce(v.begin(), v.end(), 0.0f);
assert(result == 23.0f);
```

§ 5.4 Identity may not exist or may be unknown

Not every binary operator has an identity. For instance, integers have no identity for the maximum operation. (For floating-point numbers, `-Inf` serves as an identity for maximum.) Adoption of `ranges::max_element` in [P3179R9] mitigates this, but only partially. This is because users commonly compose multiple binary operations into a single reduction. If one of those binary operations has no identity, then the composed operation does not either. The following `max_and_sum` operation that computes the maximum and sum of a range of integers is an example.

```
struct max_and_sum_result {
  std::int64_t max = 0;
  std::int64_t sum = 0;
};
```

```
struct max_and_sum {
  max_and_sum_result
  operator() (max_and_sum_result u, max_and_sum_result v) const {
    return {std::max(u.max, v.max), u.sum + v.sum};
  }

  max_and_sum_result
  operator() (max_and_sum_result u, std::int32_t y) const {
    return (*this)(u, max_and_sum_result{y, y});
  }

  max_and_sum_result
  operator() (std::int32_t x, max_and_sum_result v) const {
    return (*this)(max_and_sum_result{x, x}, v);
  }

  max_and_sum_result operator() (std::int32_t x, std::int32_t y) const {
    return (*this)(max_and_sum_result{x, x},
                   max_and_sum_result{y, y});
  }
};

template<ranges::forward_range Range>
max_and_sum_result inf_and_one_norm(Range&& r) {
  return ranges::reduce(std::forward<Range>(r), max_and_sum{});
}
```

The binary operator `max_and_sum` has no identity, because integers have no identity for the maximum operation. However, if a range is nonempty and its first element is `x_0`, `max_and_sum_result{x_0, 0}` works like an identity for the range, even though it is not an identity for the binary operator `max_and_sum`. We call this value a *pseudoidentity* of the binary operator and range. It's an interesting mathematical question whether every (mathematically associative and commutative) binary operator and nonempty range together have a pseudoidentity. Even if it does, determining a pseudoidentity might not be obvious to users. Users also might not want to access elements of the range outside of a parallel algorithm, for performance reasons.

§ 5.4.1 Do not assume that `T{}` (value-initialized `T`) is an identity

The identity value of a binary operator that returns `T` need not necessarily be `T{}` (a value-initialized `T`) for all operators and types.

— For `std::multiplies{}` it's `T(1)`.

— For "addition" in the max-plus ("tropical") semiring it's `-Inf`.

We don't want to force users to wrap reduction result types so that `T{}` defines the identity (if it exists) for `operator+(T, T)`.

— What if there is no identity or the user does not know it?

— What if `T` differs from the input range's value type?

— What if users want to use the same value type for different binary operators, such as `double` as the value type for `plus`, `multiplies`, and `ranges::max`?

— If we make users write a custom default constructor for `T`, they are more likely to make `T` not trivially constructible, and thus hinder optimizations.

Note that this differs from `std::linalg`'s algorithms, where "[a] value-initialized object of linear algebra value type shall act as the additive identity" ([linalg.reqs.val] 3). However, `std::linalg` does not take user-defined binary operators; it always uses `operator+` for reductions. Also, `std::linalg` needs "zero" for reasons other than reductions, e.g., for supporting user-defined complex number types (*`imag-if-needed`*). For these reasons, we think it's reasonable to make a different design choice for numeric range algorithms than for `std::linalg`.

§ 5.5 Initial value matters most for sequential reduction

Users who never use parallel reductions may miss the importance of the reduction identity. Let's consider typical code that sums elements of an indexed array.

```
float sum(std::span<float> a) {
  float s = 0.0f;
  for (std::size_t i = 0; i < a.size(); ++i) {
    s += a[i];
  }
  return s;
}
```

The identity element `0.0f` is used to initialize the *accumulator* into which the array's values are summed. It defines both the type of the accumulator (`float`, in this case), and its initial value. If an initial value for the reduction is provided, it replaces the identity in the code above. A serial implementation of `reduce` therefore does not need to know its binary operation's identity when an initial value is provided.

The initial value parameter of `reduce` also lets users express a "running reduction" where the whole range is not available all at once and users need to call `reduce` repeatedly. However, it is convenient but not required for that, because users already have the binary operator and the reduction result; they can always include more terms themselves without additional cost.

§ 5.6 Identity matters most for parallel reduction

The situation is different for parallel execution, because more than one accumulator must be initialized. Any parallel reduction somehow distributes the data over multiple threads of execution, where each thread uses a local accumulator for its part of the job. The initial value can be used to initialize at most one of those accumulators; for the others, something else is needed.

If an identity `id` for a binary operator `op` is known, then here is a natural way to parallelize `reduce(R, init, op)` over P processors using the serial version as a building block.

1. Partition the range R into P distinct subsequences S_p.
2. On each processor p compute a local result L_p = `reduce(`S_p`, id, op)` (with `id` as the initial value).
3. Reduce over the local results L_p with `init` as the initial value.

It's not the only and not necessarily the best way though. For example, a SIMD-based implementation for the `unseq` policy likely would not call the serial algorithm, yet it would need to initialize a local accumulator for each SIMD lane.

§ 5.7 How to initialize each local accumulator without an identity

What if the identity is unknown or does not exist? What happens to a parallel implementation of C++17 `std::reduce` with a user-defined binary operation? There are two other ways to initialize each local accumulator.

1. With some value from that subsequence, such as the first one.
2. With the result of applying the binary operation to two values from the subsequence.

The type requirements of `std::reduce` seem to assume the second approach, as the element type is not required to be convertible to the type of the result.

```
// using random access iterators for simplicity
auto sum = std::move(op(first[0], first[1]));
std::size_t sz = last - first;
for (std::size_t i = 2; i < sz; ++i) {
  sum = std::move(op(sum, first[i]));
}
```

While technically doable, this approach may be suboptimal. In many use cases, the iteration space and the data storage are aligned (e.g., to `std::hardware_constructive_interference_size` or to the SIMD width) to allow for more efficient hardware use. The loop bound changes shown above break this alignment. This may affect code efficiency.

§ 5.8 Other parallel programming models

Other parallel programming models provide all combinations of design options. Some compute only `reduce_first`, some only `reduce`, and some compute both. Some have a way to specify only an identity element, some only an initial value, and some both.

MPI (the Message Passing Interface for distributed-memory parallel communication) has reductions and lets users define custom binary operations. MPI's reductions compute the

analog of `reduce_first`. Users have no way to specify either an initial value or an identity for their custom operations.

In the Draft Fortran 2023 Standard, the `REDUCE` clause permits specification of an identity element.

OpenMP lets users specify the identity value (via an *initializer clause* `initializer(`*initializer-expr*`)`), which "determines the initializer for the private copies of list items in a reduction clause" (see Sections 7.6.2.2 and 7.6.16 of the OpenMP 6.0 specification). Per Section 7.6.6, class types used with predefined ("implicitly declared") reduction operations must satisfy one of the following two concepts: either

```
template<class T>
requires(T&& t) {
  T();
  t = 0;
};
```

or

```
template<class T>
requires() { T(0); };
```

That allows constructing a proper identity value of the class type for each predefined operation.

Kokkos lets users define the identity value for custom reduction result types, by giving the reducer class an `init(value_type& value)` member function that sets `value` to the identity (see the section on custom reducers in the Kokkos Programming Guide).

The oneTBB specification asks users to specify the identity value as an argument to `parallel_reduce` function template (see the relevant oneTBB specification page).

SYCL lets users specify the identity value by specializing `sycl::known_identity` class template for a custom reduction operation (see the relevant section of the SYCL specification).

The `std::linalg` linear algebra library in the Working Draft for C++26 says, "A value-initialized object of linear algebra value type shall act as the additive identity" ([linalg.reqs.val] 3).

In Python's NumPy library, `numpy.ufunc.reduce` takes optional initial values. If not provided and the binary operation (a "universal function" (ufunc), effectively an element-wise binary operation on a possibly multidimensional array) has an identity, then the initial values default to the identity. If the binary operation has no identity or the initial values are `None`, then this works like `reduce_first`.

§ 5.9 Implementations may use a default identity value via as-if rule

Implementations may use a default identity value for known cases, like `std::plus` or `std::multiplies` with arithmetic types.

§ 5.10 Interface for specifying identity

§ 5.10.1 Design goals

1. Allow identity as an optional optimization
2. Avoid confusion with C++17 algorithms' initial value
3. Let users specify a different identity for a given binary operation and a value type
4. Let users specify an identity even if their binary operation is a lambda
5. Let users specify a nondefault identity value "in line" with invoking the algorithm, without doing something extra (e.g., specializing a class, a trait, etc.)

Items 1 and 2 suggest that the identity should not be a separate parameter `T id` of the algorithms. That would overly emphasize an optimization hint, and it could result in confusion between C++17 numeric algorithms and our new ranges numeric algorithms.

Items 3, 4, and 5 strongly suggest that we should not rely solely on a compile-time trait for getting the identity value. Users need a way to provide the identity value at run time. (For an example of a compile-time trait system, please see the "Reduction Variables" section of the SYCL Reference. SYCL requires users to specify the identity as a `static constexpr` member of a specialization of `known_identity` for their binary operator type.)

§ 5.10.2 Design outline

Here is a prototype that shows three different designs, including this one.

1. Algorithms use a trait and a customization point to look for an optional identity in the binary operator itself.
    a. If `has_identity_value<BinaryOperator>` is `true`, the algorithm can use `identity_value<range_value_t<InRange>>(BinaryOperator)` to get the operator's identity.
    b. `identity_value` has an explicit template parameter so that it can change its behavior based on the input range's value type. For example, `binary_operation<Op, void>` (see below) returns a value-initialized value of the input range's value type.
2. We provide a binary operator wrapper `binary_operation` that lets users
    a. specify the identity value,
    b. say that the algorithm should assume that the identity does not exist, or

c. let the algorithm pick a reasonable default.

3. Users can also define their own binary operation types and customizations of `identity_value`.

The `binary_operation` wrapper is also a binary operation, just like `std::linalg`'s `layout_transpose` is a valid `mdspan` layout.

§ 5.10.2.1 `no_identity_t`: Express that an identity doesn't exist

```
struct no_identity_t {};
inline constexpr no_identity_t no_identity{};
```

The `no_identity` tag expresses that an identity value doesn't exist or isn't known for the given binary operator. Min and max on integers both have this problem (as integers lack representations of positive and negative infinity).

Having this lets us implement `ranges::min_element` and `ranges::max_element` using `ranges::reduce`.

§ 5.10.2.2 `binary_operation`: Binary operation wrapper that can hold identity too

The `binary_operation` struct holds both the binary operation, and an identity value, if the user provides one. Users can construct it in three different ways.

1. Via CTAD, by providing a binary operator and identity value

```
binary_operation bop{
  [] (auto x, auto y) { return x + y; },
  0.0
};
```

2. By specifying the template arguments and using `void` as the identity type, which tells algorithms to use a value-initialized `ranges_value_t<R>` as the identity

```
binary_operation<std::plus<void>, void> bop_void{};
```

3. By specifying the binary operation and the `no_identity` tag value, to indicate that the user wants the algorithm to assume that the binary operation has no known identity

```
binary_operation bop_no_id{my_op, no_identity};
```

A key feature of `binary_operation` is that it is a working binary operation. That is, it has a call operator and it forwards calls to the user's binary operation. This is because the identity is an optional optimization. Algorithms *could* just call `binary_operation`'s call operator and ignore the identity value, and they would get a correct answer.

Here is a sketch of the implementation of `binary_operation`. We start with a base class `binary_operation_base` that implements call operator forwarding. It prefers the user's

const call operator if it exists; this makes use of `binary_operation` in parallel algorithms easier.

```
template<class BinaryOp>
struct binary_operation_base {
  template<class Arg0, class Arg1>
  constexpr auto operator() (Arg0&& arg0, Arg1&& arg1) const
    requires std::invocable<
      std::add_const_t<BinaryOp>,
      decltype(std::forward<Arg0>(arg0)),
      decltype(std::forward<Arg1>(arg1))>
  {
    return std::as_const(op)(
      std::forward<Arg0>(arg0),
      std::forward<Arg1>(arg1));
  }

  template<class Arg0, class Arg1>
  constexpr auto operator() (Arg0&& arg0, Arg1&& arg1)
    requires (! std::invocable<
      std::add_const_t<BinaryOp>,
      decltype(std::forward<Arg0>(arg0)),
      decltype(std::forward<Arg1>(arg1))>)
  {
    return op(
      std::forward<Arg0>(arg0),
      std::forward<Arg1>(arg1));
  }

  [[no_unique_address]] BinaryOp op;
};
```

The `binary_operation` struct has two template parameters: the type of the binary operator, and the type of the identity. `Identity` can be, say, `constant_wrapper` of the value, not the actual value. This works because the accumulator type is deduced from the operator result.

```
template<class BinaryOp, class Identity>
struct binary_operation :
  public binary_operation_base<BinaryOp>
{
  [[no_unique_address]] Identity id;
};
```

We value-initialize the identity by default, if its type supports that. `Identity=no_identity_t` means that the binary operator does not have an identity, or the user does not know an identity value. It still gets "stored" in the struct so that the struct can remain an aggregate. Otherwise, it would need a one-parameter constructor for that case.

```
template<class BinaryOp, class Identity>
requires requires { Identity{}; }
struct binary_operation<BinaryOp, Identity> :
  public binary_operation_base<BinaryOp>
{
```

```
  [[no_unique_address]] Identity id{};
};
```

As with `std::plus<void>`, `Identity=void` means "the algorithm needs to deduce the identity type and value."

```
template<class BinaryOp>
struct binary_operation<BinaryOp, void> :
  public binary_operation_base<BinaryOp>
{
  [[no_unique_address]] BinaryOp op;
};
```

We define deduction guides so that algorithms by default do not assume the existence of an identity.

```
template<class BinaryOp, class Identity>
binary_operation(BinaryOp, Identity) ->
  binary_operation<BinaryOp, Identity>;

template<class BinaryOp>
binary_operation(BinaryOp) ->
  binary_operation<BinaryOp, no_identity_t>;
```

Finally, we specialize `has_identity_value` and overload `identity_value`. `Identity=void` means that `binary_operation` itself does not specify the identity type or value; rather, the algorithm must supply the type, and `identity_value` returns a value-initialized object of that type. This is why `identity_value` has a required `InputRangeValueType` template parameter.

```
template<class BinaryOp, class Identity>
constexpr bool has_identity_value<
  binary_operation<BinaryOp, Identity>> = true;

template<class BinaryOp>
constexpr bool has_identity_value<
  binary_operation<BinaryOp, no_identity_t>> = false;

template<std::default_initializable InputRangeValueType,
         class BinaryOp>
constexpr auto
identity_value(const binary_operation<BinaryOp, void>&) {
  return InputRangeValueType{};
}

template<class InputRangeValueType,
         class BinaryOp, class Identity>
  requires(! std::is_same_v<Identity, no_identity_t>)
constexpr auto
identity_value(const binary_operation<BinaryOp, Identity>& bop) {
  return bop.id;
}
```

§ 5.10.2.3 Algorithm overloads

The above infrastructure means that algorithms only need a `BinaryOp` template parameter and `binary_op` function parameter for the binary operator. Ability to use an identity value if available does not increase the number of overloads. The definitions of algorithms can use `if constexpr(has_identity_value<BinaryOp>)` to dispatch at compile time between code that uses the identity value and code that does not.

§ 5.11 Other designs

§ 5.11.1 Separate wrapped identity parameter: `op_identity<T>{value}`

In this design, users supply an identity value by wrapping it in a named struct `op_identity` and passing it in as a separate optional argument that immediately follows the binary operator to which it applies.

```
template<class Identity=void>
struct op_identity;

template<class Identity>
struct op_identity {
  [[no_unique_address]] Identity id;
};

template<std::default_initializable Identity>
struct op_identity<Identity> {
  [[no_unique_address]] Identity id{};
};

template<>
struct op_identity<void> {};

template<>
struct op_identity<no_identity_t> {};
```

The `Identity` template parameter can be `constant_wrapper` of the value, not the actual value. This works because the accumulator type is deduced from the operator result. The default template argument permits using `op_identity{}` as an argument of `exclusive_scan`. As with `binary_operation<BinaryOp, void>` above, `Identity=void` tells the algorithm to deduce the identity value as a value-initialized object of the input range's value type.

It should be rare that users need to spell out `op_identity<no_identity_t>`. Nevertheless, we include an abbreviation `no_op_identity` to avoid duplicate typing.

```
inline constexpr op_identity<no_identity_t> no_op_identity{};
```

We define a customization point `identity_value` analogously to the way we defined it with the `binary_operation` design above.

```
template<class InputRangeValueType, class Identity>
  requires(! std::is_same_v<Identity, no_identity_t>)
constexpr auto identity_value(op_identity<Identity> op_id) {
```

```
  return op_id.id;
}

template<std::default_initializable InputRangeValueType>
constexpr auto identity_value(op_identity<void>) {
  return InputRangeValueType{};
}
```

Users would have two ways to provide a nondefault identity value.

1. Construct `op_identity` with a default value using aggregate initialization: `op_identity{nondefault_value}`

2. Specialize `op_identity<T>` so `declval<op_identity<T>>().value` is the value

For example, users could inherit their specialization from `constant_wrapper`.

```
namespace impl {
  inline constexpr my_number some_value = /* value goes here */;
}
template<class T>
struct op_identity<my_number> :
  constant_wrapper<impl::some_value>
{};
```

Here are some use cases.

```
// User explicitly opts into "most negative integer"
// as the identity for min.  This should not be the default,
// as the C++ Standard Library has no way to know
// whether this represents a valid input value.
constexpr auto lowest = std::numeric_limits<int>::lowest();
auto result5 = std::ranges::reduce(exec_policy, range,
  std::ranges::min, reduce_identity{lowest});

// range_value_t<R> is float, but identity value is double
// (even though it's otherwise the default value, zero).
// std::plus<void> should use operator()(double, double) -> double
auto result6 = std::ranges::reduce(exec_policy, range,
  std::plus{}, reduce_identity{0.0});
```

Advantages of this approach:

— Users would see in plain text the purpose of this function argument

— Algorithms could overload on it without risk of ambiguity

— The struct is an aggregate, which would maximize potential for optimizations

— It would not impose requirements on the user's binary function

Disadvantages:

— The algorithm could not use this to deduce a default identity value from a binary operation

— A specialization of `op_identity<T>` would take effect for all binary operations on `T`

§ 5.12 If users can define an identity value, do they need an initial value?

§ 5.12.1 `*reduce` algorithms should not take both

— Providing both would confuse users and would specify the result type redundantly.

— There is no performance benefit for providing an initial value, if an identity value is known.

```
std::vector<int> v{5, 11, 7};
const int max_identity = std::numeric_limits<int>::lowest();

// identity as initial value
int result1 = ranges::reduce(v, max_identity, ranges::max{});
assert(result1 == 11);

// identity as, well, identity
int result2 = ranges::reduce(v,
  reduce_operation{ranges::max{}, max_identity});
assert(result2 == 11);

std::vector<int> empty_vec;
int result3 = ranges::reduce(empty_vec,
  reduce_operation{ranges::max{}, max_identity});
assert(result3 == max_identity);
```

§ 5.12.1.1 `*_scan` algorithms would benefit from an initial value

— Initial value affects every element of output

— Without it, would need extra `transform` pass over output

— For exclusive scan, can't use `transform_exclusive_scan` to work around non-identity initial value

```
std::vector<int> in{5, 7, 11, 13, 17};
std::vector<int> out(std::size_t(5));
constexpr int init = 3;
auto binary_op = std::plus{};

// out: 8, 15, 26, 39, 56
ranges::inclusive_scan(in, out, binary_op, init);

// out: 3, 8, 15, 26, 39
// Yes, init and binary_op have reversed order.
ranges::exclusive_scan(in, out, init, binary_op);
```

```
// out: 8, 15, 26, 39, 56
auto unary_op = [op = binary_op] (auto x) { return op(x, 3); };
ranges::transform_inclusive_scan(int, out, binary_op, unary_op);

// out: 0, 8, 15, 26, 39
ranges::transform_exclusive_scan(in, out, binary_op, unary_op);
```

5.12.1.2 Avoid mixing up identity and initial value

C++17 *reduce and *_scan take initial value `T init`, undecorated.

If new algorithms take `T identity`, then users could be confused when switching from C++17 to new algorithms.

"Decorating" identity by wrapping it in a struct prevents confusion. It also lets algorithms provide both initial value and identity.

```
std::vector<int> in{-8, 6, -4, 2, 0, 10, -12};
std::vector<int> out(std::size_t(7));
constexpr int init = 7;
auto binary_op = std::ranges::max{};

// for inclusive_scan an initial value can be omitted.

// out: -8, 6, 6, 6, 6, 10, 10
std::ranges::inclusive_scan(in, out, binary_op);

// out: 7, 7, 7, 7, 7, 10, 10
std::ranges::inclusive_scan(in, out, binary_op, init);

// Suppose the user knows that they
// will never see values smaller than -9.
const int identity_value = -10;

// out: 7, 7, 7, 7, 7, 10, 10
std::ranges::inclusive_scan(in, out,
  reduce_operation{binary_op, identity_value},
  init);

// exclusive scan requires an initial value.
// Identity is a reasonable default initial value,
// if you have it.
//
// C++17 *exclusive_scan puts init left of binary_op,
// while inclusive_scan puts init right of binary_op.
// We find this weird so we don't do it.

// out: 7, 7, 7, 7, 7, 7, 10
std::ranges::exclusive_scan(in, out, binary_op, init);

// out: -10, -8, 6, 6, 6, 6, 10
std::ranges::exclusive_scan(in, out,
  reduce_operation{binary_op, identity_value});
```

```
// out: 7, 7, 7, 7, 7, 7, 7, 10
std::ranges::exclusive_scan(in, out,
  reduce_operation{binary_op, identity_value}, init);
```

§ 5.13 Conclusions

It's important for both performance and functionality that users be able to specify an identity value for parallel reductions. Designs for this should avoid confusion when switching from C++17 parallel numeric algorithms to the new ranges versions. We would like feedback from SG9 and LEWG on their preferred design.

Our proposed `*reduce` algorithms do not need an initial value parameter. For our proposed `*_scan` algorithms, an initial value could improve performance in some cases by avoiding an additional pass over all the output elements. The `*exclusive_scan` algorithms need an initial value because it defines the first element of the output range. The initial value could default to the identity, if it exists and is known.

§ 6 Implementation

The oneAPI DPC++ library (oneDPL) has a partial deployment experience. The implementation is done as experimental with the following deviations from this proposal:

- Algorithms do not have constraints
- Design for optional identity is not implemented1
- `reduce` has more overloads (without init and without binary predicate)
- `*_scan` return type is not `in_out_result`
- The convenience wrappers proposed in this paper are not implemented. Their implementation is expected to be trivial, though.

§ 7 Wording

Note: the whole section is WORK IN PROGRESS.

> Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

§ 7.1 Update feature test macro

> In [version.syn], increase the value of the `__cpp_lib_parallel_algorithm` macro by replacing YYYYMML below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_parallel_algorithm YYYYMML // also in <algorithm>
```

§ 7.2 Add *sized-forward-range* to [range.refinements]

7 The `constant_range` concept specifies the requirements of a `range` type whose elements are not modifiable.

```
template<class T>
  concept constant_range =
    input_range<T> && constant-iterator<iterator_t<T>>;
```

8 The exposition-only concept *sized-forward-range* specifies the requirements of a `range` type that is sized and whose iterators model `forward_iterator`.

```
template<class T>
  concept sized-forward-range =           // exposition only
    forward_range<R> && sized_range<R>;
```

9 The exposition-only concept *sized-random-access-range* specifies the requirements of a `range` type that is sized and allows random access to its elements.

```
template<class T>
  concept sized-random-access-range =           // exposition only
    random_access_range<R> && sized_range<R>;
```

[*Note 1*: ~~This concept~~The concepts *sized-forward-range* and *sized-random-access-range* constrain~~s~~ some parallel algorithm overloads; see [algorithms] and [numeric]. – *end note*]

§ 7.3 Change [numeric.ops.overview]

Change [numeric.ops.overview] (the <numeric> header synopsis) as follows.

§ 7.3.1 Add declaration of exposition-only concepts

Add declarations of exposition-only concepts *indirectly-binary-foldable-impl* and *indirectly-binary-foldable* to [numeric.ops.overview] (the <numeric> header synopsis) as follows.

Note that the exposition-only concepts *indirectly-binary-left-foldable* and *indirectly-binary-right-foldable* live in the <algorithm> header.

```
// mostly freestanding
namespace std {
```

```
namespace ranges {

  template<class F, class T, class I, class U>
    concept indirectly-binary-foldable-impl =       // exposition only
      movable<T> && movable<U> &&
      convertible_to<T, U> &&
      invocable<F&, U, iter_reference_t<I>> &&
      assignable_from<U&, invoke_result_t<F&, U, iter_reference_t<I>>> &&
```

```
    invocable<F&, iter_reference_t<I>, U> &&
    assignable_from<U&, invoke_result_t<F&, iter_reference_t<I>, U>>;
template<class F, class T, class I>
  concept indirectly-binary-foldable =           // exposition only
    copy_constructible<F> && indirectly_readable<I> &&
    invocable<F&, T, iter_reference_t<I>> &&
    convertible_to<invoke_result_t<F&, T, iter_reference_t<I>>,
      decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>> &&
    invocable<F&, iter_reference_t<I>, T> &&
    convertible_to<invoke_result_t<F&, iter_reference_t<I>, T>,
      decay_t<invoke_result_t<F&, iter_reference_t<I>, T>>> &&
    indirectly-binary-foldable-impl<F, T, I,
      decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>>;
}
```

```
// [accumulate], accumulate
template<class InputIterator, class T>
    constexpr T accumulate(InputIterator first, InputIterator last, T
      init);
template<class InputIterator, class T, class BinaryOperation>
    constexpr T accumulate(InputIterator first, InputIterator last, T
      init,
                           BinaryOperation binary_op);
```

§ 7.3.2 Add declarations of `reduce`, `sum`, `product`, and their `*_into` variants

Add declarations of ranges overloads of `reduce`, `reduce_into`, `sum`, `sum_into`, `product`, and `product_into` algorithms to [numeric.ops.overview] (the <numeric> header synopsis) as follows.

```
template<class ExecutionPolicy, class ForwardIterator, class T, class
    BinaryOperation>
  T reduce(ExecutionPolicy&& exec, // freestanding-deleted, see
    [algorithms.parallel.overloads]
             ForwardIterator first, ForwardIterator last, T init,
    BinaryOperation binary_op);
```

```
namespace ranges {

// Non-parallel overloads of reduce

template<forward_iterator I,
         sized_sentinel_for<I> S,
         class T = iter_value_t<I>,
         indirectly-binary-foldable<T, I> F>
  constexpr auto reduce(I first, S last, T init, F binary_op) -> /* see
    below */;
template<sized-forward-range R,
         class T = range_value_t<R>,
         indirectly-binary-foldable<T, iterator_t<R>> F>
  constexpr auto reduce(R&& r, T init, F binary_op) -> /* see below */;

// Parallel overloads of reduce

template<execution-policy Ep,
```

```
          random_access_iterator I,
          sized_sentinel_for<I> S,
          class T = iter_value_t<I>,
          indirectly-binary-foldable<T, I> F>
            auto   reduce(Ep&&   exec,   //   freestanding-deleted,   see
      [algorithms.parallel.overloads]
                I first, S last, T init, F binary_op) -> /* see below */;
template<execution-policy Ep,
          sized-random-access-range R,
          class T = range_value_t<R>,
          indirectly-binary-foldable<T, iterator_t<R>> F>
            auto   reduce(Ep&&   exec,   //   freestanding-deleted,   see
      [algorithms.parallel.overloads]
                R&& r, T init, F binary_op) -> /* see below */;

// Non-parallel overloads of reduce_into

template<forward_iterator I,
          sized_sentinel_for<I> IS,
          forward_iterator O,
          sized_sentinel_for<O> OS,
          class T = iter_value_t<I>,
          indirectly-binary-foldable<T, I> F>
  constexpr auto reduce_into(
    I in_first, IS in_last,
    O out_first, OS out_last,
    T init,
    F binary_op) -> /* see below */;
template<sized-forward-range IR,
          sized-forward-range OR,
          class T = range_value_t<IR>,
          indirectly-binary-foldable<T, iterator_t<IR>> F>
  constexpr auto reduce_into(
    IR&& in_range,
    OR&& out_range,
    T init,
    F binary_op) -> /* see below */;

// Parallel overloads of reduce_into

template<execution-policy Ep,
          random_access_iterator I,
          sized_sentinel_for<I> S,
          forward_iterator O,
          sized_sentinel_for<O> OS,
          class T = iter_value_t<I>,
          indirectly-binary-foldable<T, I> F>
          auto   reduce_into(Ep&&   exec,   //   freestanding-deleted,   see
      [algorithms.parallel.overloads]
    I in_first, S in_last,
    O out_first, OS out_last,
    T init,
    F binary_op) -> /* see below */;
template<execution-policy Ep,
          sized-random-access-range IR,
          sized-forward-range OR,
```

```
          class T = range_value_t<IR>,
          indirectly-binary-foldable<T, iterator_t<IR>> F>
        auto   reduce_into(Ep&&   exec,   //   freestanding-deleted,   see
      [algorithms.parallel.overloads]
    IR&& in_range,
    OR&& out_range,
    T init,
    F binary_op) -> /* see below */;

// Non-parallel overloads of sum

template<forward_iterator I,
         sized_sentinel_for<I> S>
  requires /* see below */
    constexpr auto sum(I first, S last) -> /* see below */;
template<sized-forward-range R>
  requires /* see below */
    constexpr auto sum(R&& r) -> /* see below */;

// Parallel overloads of sum

template<execution-policy Ep,
         random_access_iterator I,
         sized_sentinel_for<I> S>
  requires /* see below */
              auto   sum(Ep&&   exec,   //   freestanding-deleted,   see
      [algorithms.parallel.overloads]
             I first, S last) -> /* see below */;
template<execution-policy Ep,
         sized-random-access-range R>
  requires /* see below */
              auto   sum(Ep&&   exec,   //   freestanding-deleted,   see
      [algorithms.parallel.overloads]
             R&& r) -> /* see below */;

// Non-parallel overloads of sum_into

template<forward_iterator I,
         sized_sentinel_for<I> IS,
         forward_iterator O,
         sized_sentinel_for<O> OS>
  requires /* see below */
    constexpr auto sum_into(I in_first, S in_last,
      O out_first, OS out_last) -> /* see below */;
template<sized-forward-range IR,
         sized-forward-range OR>
  requires /* see below */
    constexpr auto sum_into(IR&& input, OR&& output) -> /* see below */;

// Parallel overloads of sum_into

template<execution-policy Ep,
         random_access_iterator I,
         sized_sentinel_for<I> IS,
         forward_iterator O,
         sized_sentinel_for<O> OS>
```

```
  requires /* see below */
           auto  sum_into(Ep&&  exec,  //  freestanding-deleted,  see
      [algorithms.parallel.overloads]
      I in_first, IS in_last,
      O out_first, OS out_last) -> /* see below */;
template<execution-policy Ep,
         sized-random-access-range IR,
         sized-forward-range OR>
  requires /* see below */
           auto  sum_into(Ep&&  exec,  //  freestanding-deleted,  see
      [algorithms.parallel.overloads]
      OR&& out,
      IR&& in) -> /* see below */;

// Non-parallel overloads of product

template<forward_iterator I,
         sized_sentinel_for<I> S>
  requires /* see below */
    constexpr auto product(I first, S last) -> /* see below */;
template<sized-forward-range R>
  requires /* see below */
    constexpr auto product(R&& r) -> /* see below */;

// Parallel overloads of product

template<execution-policy Ep,
         random_access_iterator I,
         sized_sentinel_for<I> S>
  requires /* see below */
            auto  product(Ep&&  exec,  //  freestanding-deleted,  see
      [algorithms.parallel.overloads]
                I first, S last) -> /* see below */;
template<execution-policy Ep,
         sized-random-access-range R>
  requires /* see below */
            auto  product(Ep&&  exec,  //  freestanding-deleted,  see
      [algorithms.parallel.overloads]
                R&& r) -> /* see below */;

// Non-parallel overloads of product_into

template<forward_iterator I,
         sized_sentinel_for<I> IS,
         forward_iterator O,
         sized_sentinel_for<O> OS>
  requires /* see below */
    constexpr auto product_into(
      I in_first, IS in_last,
      O out_first, OS out_last) -> /* see below */;
template<sized-forward-range IR,
         sized-forward-range>
  requires /* see below */
    constexpr auto product_into(
      IR&& in,
      OR& out) -> /* see below */;
```

```
  // Parallel overloads of product_into

  template<execution-policy Ep,
           random_access_iterator I,
           sized_sentinel_for<I> IS,
           forward_iterator O,
           sized_sentinel_for<O> OS>
    requires /* see below */
             auto  product_into(Ep&&  exec,  //  freestanding-deleted,  see
        [algorithms.parallel.overloads]
        I in_first, IS in_last,
        O out_first, OS out_last) -> /* see below */;
  template<execution-policy Ep,
           sized-random-access-range IR,
           sized-forward-range OR>
    requires /* see below */
             auto  product_into(Ep&&  exec,  //  freestanding-deleted,  see
        [algorithms.parallel.overloads]
        IR&& in,
        OR&& out) -> /* see below */;

  } // namespace ranges
```

```
// [inner.product], inner product
template<class InputIterator1, class InputIterator2, class T>
  constexpr T inner_product(InputIterator1 first1, InputIterator1 last1,
                            InputIterator2 first2, T init);
```

§ 7.3.3 Add declarations of ranges `transform_reduce`, `dot`, and `dot_into`

```
  template<class ExecutionPolicy, class ForwardIterator, class T,
           class BinaryOperation, class UnaryOperation>
      T  transform_reduce(ExecutionPolicy&& exec, // freestanding-deleted,
        see [algorithms.parallel.overloads]
                          ForwardIterator first, ForwardIterator last, T
        init,
                               BinaryOperation binary_op, UnaryOperation
        unary_op);
```

```
  namespace ranges {

  // Non-parallel overloads of unary transform_reduce

  // TODO

  // Parallel overloads of unary transform_reduce

  // TODO

  // Non-parallel overloads of unary transform_reduce_into

  // TODO

  // Parallel overloads of unary transform_reduce_into
```

```
// TODO

// Non-parallel overloads of binary transform_reduce

// TODO

// Parallel overloads of binary transform_reduce

// TODO

// Non-parallel overloads of binary transform_reduce_into

// TODO

// Parallel overloads of binary transform_reduce_into

// TODO

// Non-parallel overloads of dot

// TODO

// Parallel overloads of dot

// TODO

// Non-parallel overloads of dot_into

// TODO

// Parallel overloads of dot_into

// TODO

}
```

```
// [partial.sum], partial sum
template<class InputIterator, class OutputIterator>
  constexpr OutputIterator
    partial_sum(InputIterator first, InputIterator last,
                OutputIterator result);
```

§ 7.3.4 Add declarations of ranges `exclusive_scan`

```
    template<class ExecutionPolicy, class ForwardIterator1, class
      ForwardIterator2, class T,
         class BinaryOperation>
  ForwardIterator2
     exclusive_scan(ExecutionPolicy&& exec,                    // free-
      standing-deleted, see [algorithms.parallel.overloads]
                  ForwardIterator1 first, ForwardIterator1 last,
                       ForwardIterator2 result, T init, BinaryOperation
      binary_op);
```

```
namespace ranges {
  // Non-parallel overloads of exclusive_scan
```

```
  // TODO

  // Parallel overloads of exclusive_scan

  // TODO
}
```

```
// [inclusive.scan], inclusive scan
template<class InputIterator, class OutputIterator>
  constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                   OutputIterator result);
```

§ 7.3.5 Add declarations of ranges `inclusive_scan`

```
    template<class ExecutionPolicy, class ForwardIterator1, class
      ForwardIterator2,
         class BinaryOperation, class T>
  ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec,                    // free-
      standing-deleted, see [algorithms.parallel.overloads]
                   ForwardIterator1 first, ForwardIterator1 last,
                   ForwardIterator2 result, BinaryOperation binary_op, T
      init);
```

```
namespace ranges {
  // Non-parallel overloads of inclusive_scan

  // TODO

  // Parallel overloads of inclusive_scan

  // TODO
}
```

```
// [transform.exclusive.scan], transform exclusive scan
template<class InputIterator, class OutputIterator, class T,
         class BinaryOperation, class UnaryOperation>
  constexpr OutputIterator
    transform_exclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result, T init,
                               BinaryOperation binary_op, UnaryOperation
      unary_op);
```

§ 7.3.6 TODO What about `transform_exclusive_scan`?

§ 7.3.7 TODO What about `transform_inclusive_scan`?

§ 7.3.8 Add wording for algorithms

TODO

§ 8 Acknowledgements

Thanks to Bryce Adelstein Lelbach and Abhilash Majumder for a valuable contribution to earlier revisions of this paper.

§ 9 References

[P1673R13] Mark Hoemmen, Daisy Hollman,Christian Trott,Daniel Sunderland,Nevin Liber,Alicia KlinvexLi-Ta Lo,Damien Lebrun-Grandie,Graham Lopez,Peter Caday,Sarah Knepper,Piotr Luszczek,Timothy Costa. 2023-12-18. A free function linear algebra interface based on the BLAS.

https://wg21.link/p1673r13

[P1813R0] Christopher Di Bella. 2019-08-02. A Concept Design for the Numeric Algorithms.

https://wg21.link/p1813r0

[P2214R2] Barry Revzin, Conor Hoekstra, Tim Song. 2022-02-18. A Plan for C++23 Ranges.

https://wg21.link/p2214r2

[P2248R8] Giuseppe D'Angelo. 2024-03-20. Enabling list-initialization for algorithms.

https://wg21.link/p2248r8

[P2322R6] Barry Revzin. 2022-04-22. ranges::fold.

https://wg21.link/p2322r6

[P2760R1] Barry Revzin. 2023-12-15. A Plan for C++26 Ranges.

https://wg21.link/p2760r1

[P2769R3] Ruslan Arutyunyan, Alexey Kukanov. 2024-10-16. get_element customization point object.

https://wg21.link/p2769r3

[P2902R2] Oliver Rosten. 2025-05-12. constexpr "Parallel" Algorithms.

https://wg21.link/p2902r2

[P3179R9] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2025-05-29. C++ parallel range algorithms.

https://wg21.link/p3179r9

[P3351R2] Yihe Li. 2025-01-12. views::scan.

https://wg21.link/p3351r2

[P3490R0] Alexey Kukanov, Ruslan Arutyunyan. 2024-11-14. Justification for ranges as the output of parallel range algorithms.

https://wg21.link/p3490r0